

队伍编号	MC25008279
赛道	C

基于多目标优化与熵权法赋权的自适应编码去噪策略

摘要

在数字媒体技术蓬勃发展的时代，音频处理在众多领域的重要性日益凸显，高效存储、降噪处理和音色优化成为该领域面临的核心挑战。本研究聚焦这些难题，综合运用多目标优化、自适应编码等多种数学方法与技术，成功实现对音频处理关键问题的有效解决。

在问题一中，核心任务是量化不同音频格式在存储效率与音质保真度之间的平衡关系。为此，构建了综合得分模型，该模型考虑文件大小、音质损失、编解码复杂度和适用场景等因素，各因素权重需通过熵权法和层次分析法结合方法确定，最终确定权重向量 $W=[0.2465,0.304,0.254,0.1955]$ ，其中 w_1 、 w_2 、 w_3 、 w_4 分别对应文件大小、音质损失、编解码复杂度和适用场景的权重。借助 Python 进行数据处理与可视化呈现，成功设计出量化不同音频格式在存储效率与音质保真度之间平衡关系的综合评价指标。

问题二聚焦于分析采样率、比特深度、压缩算法等参数对音频质量和文件大小的影响。采用多目标优化方法，先基于音质评分脚本建立量化模型，再通过回归分析确定参数的边际效益。构建的性价比指标 CPI 综合权衡了音质、存储效率和计算复杂度，依据该指标对不同参数组合的文件进行分层排序，最终得出语音类型最佳方案为语音——44100Hz 192kbps ACC；音乐类型最佳方案为音乐——48000Hz 16bit WAV。

问题三旨在设计自适应编码方案，以打破传统固定编码参数的局限。通过提取音频特征构建目标函数，将提升音质（增加信噪比）和减小文件大小作为优化方向，运用粒子群优化（PSO）、深度 Q 网络（DQN）和支持向量机（SVM）算法优化编码参数，其中 SVM 用于精准预测音频类型，辅助选择最优编码参数。实验结果显示，音乐文件采用 WAV 格式，文件大小 680KB，信噪比达 +92.5dB；语音文件采用 MP3 格式，文件大小 96KB，信噪比为+38.7dB。与固定参数编码方案相比，该优化方案在音乐文件和语音文件上压缩节省分别达 53.1%和 50.0%，信噪比提升分别为+68.4dB 和 +21.9dB，显著优化了音频的存储与音质表现。

问题四针对带噪音频样本处理展开。先将音频分段，依据梅尔频谱熵等特征区分语音与噪声，利用频谱熵等参数和动态阈值识别。针对背景噪声、带状噪声和突发噪声，分别采用带通滤波、中值滤波和谱减法处理，处理后合并生成去噪音频。处理后的音频，part1 去噪后信噪比为 46.37dB，part2 为 49.14dB。同时分析在不同噪声类型和强度下的适用范围与局限性，形成一套完整的“识别-处理-评估”闭环去噪方案。

综上所述，本研究利用多种数学方法有效解决了音频处理中的关键问题，为音频处理领域提供了切实可行的优化策略，在相关实际场景中具有广阔的推广应用前景。

关键词：音频处理、综合评价指标、多目标优化、自适应编码、去噪

目录

一、 问题重述.....	1
1.1 问题背景	1
1.2 问题要求	1
二、 问题分析.....	2
2.1 问题一的分析	2
2.2 问题二的分析	2
2.3 问题三的分析	3
2.3.1 问题四的分析	4
三、 模型假设.....	4
四、 符号说明.....	4
五、 的建立与求解.....	5
5.1 问题一模型的建立与求解	5
5.1.1 基于归一化处理方法的综合评价模型	5
5.1.2 各子指标的归一化处理	5
5.1.3 基于层次分析法和熵权法的权重确定	6
5.2 问题二模型的建立与求解	7
5.2.1 基于多参数融合的场景适配综合性价比评分模型	8
5.2.2 排序结果以及最佳参数推荐	12
5.3 问题三模型的建立与求解	13
5.3.1 基于音频特征驱动的自适应编码优化模型	13
5.3.2 优化后参数选择、文件大小、音质保真度与方案的改进比较	14
5.4 问题四模型的建立与求解	16
5.4.1 基于时频特征融合与噪声类型识别的自适应音频去噪模型	16
5.4.2 噪声种类、去噪后信噪比以及适用范围、局限性分析	17
六、 模型的分析、评价与改进.....	21
6.1 问题一模型的分析改进	21
6.1.1 归一化算法优势	21
6.1.2 归一化算法局限性	21
6.2 模型的改进	21
6.2.1 问题三 SVM 模型的弊端与改进	22
6.2.2 SVM 模型局限性	22
6.2.3 SVM 局限性	22
附录.....	24

一、问题重述

1.1 问题背景

在数字媒体快速发展的背景下，音频处理作为信息时代的核心技术之一，正面临高效存储、降噪处理与音色优化三大关键挑战。这些问题贯穿音频从采集到呈现的全过程，亟待系统性解决，以达到提升声音质量与用户体验的目的。

音频数据往往占据大量存储空间，在音频信息的存储与传输过程中，如何在不牺牲音质的前提下压缩体积，是当前亟需解决的问题。各类音频格式（如 WAV、MP3、FLAC 等）因编码方式不同，在音质保留与文件大小之间形成了复杂的权衡关系。影响音质的关键参数如采样频率、位深及压缩算法，在不同类型的音频素材（如语音、音乐、环境音等）中表现差异显著，很难以统一标准进行参数配置。

此外，实际录制中音频常被多种背景噪声干扰，如室内的设备运转声、室外的交通与风声，甚至录音设备自身的电流噪声等，这些杂音会削弱音频的清晰度与可辨识度。由于不同类型的噪声在时域和频域上表现各异，去噪处理需依据其特性进行有针对性的算法设计，才能实现有效抑制其干扰。

附件 1 汇总了多种音频格式的详细参数及其在编码方式和存储效率方面的数据；附件 2 提供了不同噪声环境下的实录音频，涵盖从轻度干扰到严重噪声的多种场景。我们需要基于所给数据，构建数学模型，解决音频的存储结构优化、去噪等问题，从而提升整体音质表现。

1.2 问题要求

基于上述背景信息我们需要建立数学模型解决以下问题：

问题一：构建一个综合评价指标来量化主流音频格式（至少包括 WAV、MP3、AAC 这三种）在存储效率与音质保真度之间的平衡关系。该指标应综合反映文件体积、音质损耗、编解码所需计算资源，以及在不同应用场景下的适用性，如流媒体播放与高质量录音等，以实现对音频格式性能的全面评估与优选。

问题二：基于附件 1 提供的音频样本，建立数学模型，定量分析采样率、位深度及压缩算法等关键参数对音频质量与文件体积的综合影响。在此基础上，构建衡量音质与存储效率平衡关系的性价比指标，并对附件 1 中不同参数配置下生成的音频文件进行分类排序（分别针对音乐类和语音类，不含原始文件）。最后分别提出适用于音乐和语音内容的最优参数设置方案，为不同应用需求提供理论参考。

问题三：构建一套能够基于音频内容的类型（语音或音乐）、频谱结构及动态范围等特征，智能匹配最优编码参数组合的自适应编码方案，并将该方案应用于附件 1 中的原始语音与音乐样本，记录自动选择后的参数配置、文件体积及音质保真度表现，通过与固定参数方案相比较，评估所构建模型在压缩效率与音质保持方面的改进效果。

问题四：针对附件 2 中提供的音频样本，开展时频域特征分析，构建数学模型以识别并量化背景噪声、突发噪声、带状噪声等不同类型噪声的特征参数。在此基础上，设计一种能够根据噪声类型智能匹配最优处理方式的改进去噪策略或自适应算法。要求对文件中每段样本音频标注所含噪声类型，给出去噪处理后的信噪比（SNR），并分析该方法在不同噪声强度与类别下的适用性与局限性，最后将去噪后的结果存储为新的 wav 文件，并分别命名为 part1_denoised.wav 和 part2_denoised.wav 进行提交。

二、问题分析

2.1 问题一的分析

问题一要求我们设计一个综合评价指标，衡量不同音频格式（如 WAV、MP3、AAC）在**存储效率**和**音质保真度**之间的平衡，此指标有助于根据应用需求合理选择音频格式。评价时需综合考虑**文件大小**、**音质损失**和**编解码复杂度**，因为分别会影响存储和传输效率、听觉体验和设备运行效率，当然，我们还要考虑**适用场景**，因为不同场景对音频格式有不同要求，如流媒体传输注重实时性和小文件，专业录音注重音质。

基于以上问题，团队构建了一个综合评价模型，通过**归一化处理**各因素后计算综合得分，各指标**权重**可通过**熵权法结合层次分析法**确定，音质损失通过 SNR 衡量，编解码复杂度通过计算时间复杂度评估。然后使用 Python 代码实现数据处理、权重计算和得分计算。最终得到综合评价模型，它帮助比较不同音频格式的平衡表现，为实际应用提供选择依据。

上述模型框架可简化为以下流程图：

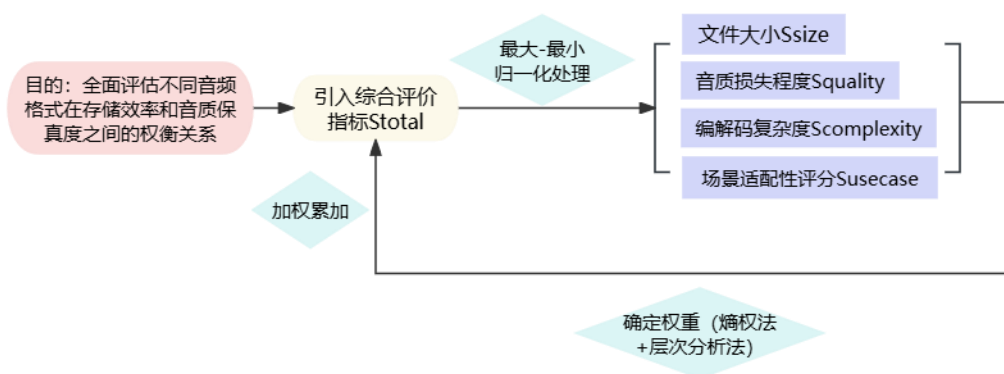


图 1：问题一模型流程图

2.2 问题二的分析

问题二要求对不同音频格式的存储效率（文件大小）与音质保真度（基于采样率、比特深度、压缩算法等参数度量）进行综合评价，并设计**性价比指标**，从而为语音和音乐两种类型的音频推荐**最佳编码参数**。分析题意与附件 1 中的数据可知，音频参数的优化需要权衡音质评分、文件大小以及适用场景等因素，大致可概括为：

1、文件大小与音质的关系：当音质评分提升幅度显著高于文件大小增长时（如语音从 8kHz 升至 16kHz），建议提高采样率，因此有设计性价比指标的必要性；

2、采样率与比特深度的选择：对于语音内容，较低的采样率和比特深度即可满足需求，而音乐内容需要更高的采样率和比特深度以保证音质；

3、适用场景的适配：流媒体传输（以语音为主）优先考虑文件大小，专业录音（以音乐为主）则优先考虑音质保真度。因此语音内容可接受更低参数组合（如 AAC/16kHz/16bit），而音乐内容可能需要更高配置（如 WAV/44.1kHz/24bit）。

我们选择主要使用**多目标优化方法**，首先基于音质评分脚本建立量化模型，定义采样率得分、位深得分和编码得分的加权计算公式；接着通过回归分析确定各参数对音质和文件大小的边际效益；然后**构建性价比指标 CPI**，综合评估音质、存储效率和计算复杂度；最后采用分层排序法，根据音频类型（语音/音乐）分别输出参数推荐方

案。针对特殊需求场景，引入敏感性分析调整权重系数，确保推荐结果的适应性。整个分析过程通过脚本生成的箱线图、散点图和折线图进行可视化验证，保证结论的直观性和可靠性。

上述模型框架可以概括为下流程图：

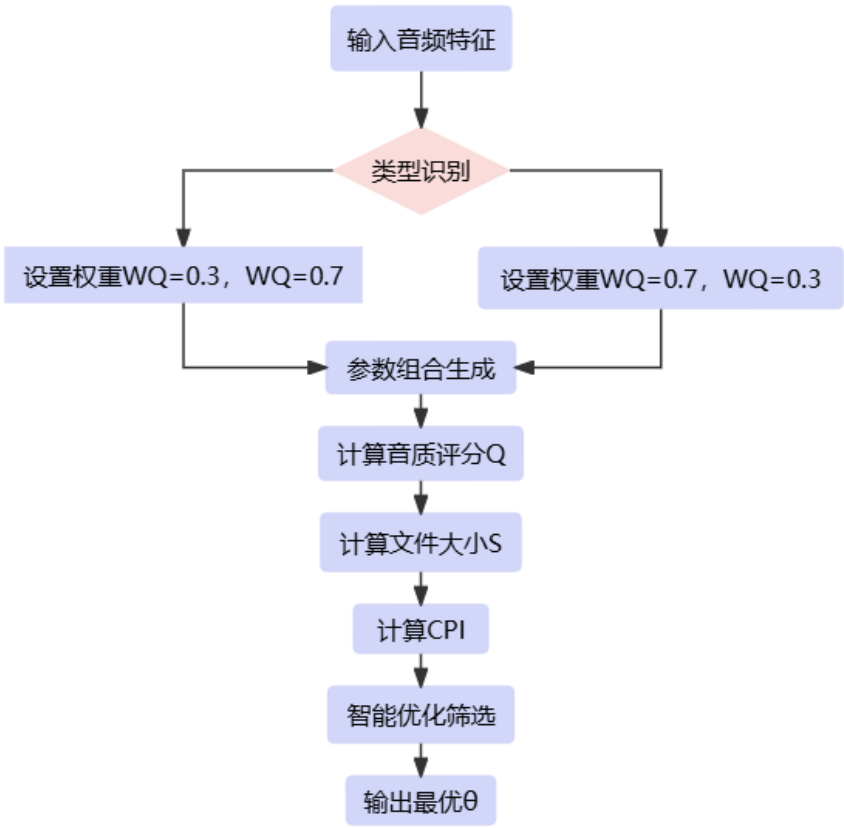


图 2：问题二模型流程图

2.3 问题三的分析

问题三要求我们设计一种自适应编码方案，以优化音频文件的音质与文件大小。传统的固定编码参数无法适应不同音频的特性，影响编码效率和音质，题干中提出了根据音频特征动态调整编码参数的方案，旨在实现音质与文件大小的最佳平衡。分析题意与数据后，我们可以将优化目标概括为：

1、提升音质：通过调整采样率、比特深度和压缩格式等参数，增加信噪比(SNR)，从而改善音质。

2、减小文件大小：通过调整压缩率，特别是在语音音频中，使用较高的压缩比来减小文件体积，同时保持语音清晰度。

团队选择通过提取音频特征并构建目标函数，平衡音质与文件大小。并使用粒子群优化 (PSO)、深度 Q 网络 (DQN) 和支持向量机 (SVM) 算法，基于音频特征优化编码参数，提升音质并减小文件大小。**SVM 模型**在音频类型预测方面发挥了重要作用，能够更准确地分类语音与音乐类型，从而为每种类型选择最优编码参数。

总的来说，我们设计的自适应编码流程先是**提取音频特征并利用分类器 (SVM) 预测音频类型**（语音或音乐），然后**根据音频类型选择合适的编码参数**，以优化音质和文件大小。

2.3.1 问题四的分析

问题四要求针对附件 2 中的带噪音频样本进行全面分析与处理，需基于时频特征构建数学模型，识别噪声类型，制定针对性去噪策略，并评估去噪效果。综合题意，本题可从**噪声分析、去噪处理与效果评估**三个层面展开：

首先是噪声分析，团队选择将音频**分段处理**，通过梅尔频谱熵、能量等特征与阈值判定区分语音与噪声；针对非语音段，计算频谱熵、频域集中度等特征参数，并采用动态阈值方法识别背景噪声、带状噪声和突发噪声，完成噪声类型分类。

其次是去噪处理，针对不同噪声类型匹配合适方法。**带状**噪声用**带通滤波**剔除特定频段；**突发**噪声用**中值滤波**去除脉冲干扰；**背景**噪声用**谱减法**降低底噪。遍历各段应用策略后进行系统综合，最终生成去噪音频。

最后是效果评估，我们选择以**信噪比（SNR）**为核心指标，结合波形图、时频图、信噪比曲线和噪声类型分布图进行可视化分析，全面呈现去噪效果，并分析不同噪声类型和强度下的适用范围与局限性。

整体流程通过自定义函数与循环结构实现，形成了一套全面的“**识别-处理-评估**”闭环的音频去噪方案。

三、 模型假设

- 1、假设题目所给音频数据都比较准确且具有代表性。
- 2、问题二中在计算“基础性价比”与“类型适配评分”时，认为音质评分与文件大小为相互独立的变量，忽略其中潜在的非线性耦合关系。
- 3、假设信噪比（SNR）可以较好地量化音频主观听感质量。
- 4、假设 PSO、DQN 等优化算法在合理迭代次数下可搜索到满足音质-文件大小平衡的参数组合。

四、 符号说明

符号	说明	单位
S_i	第 i 个样本的采样率	Hz
b_i	第 i 个样本的位深	bit
c_i	第 i 个样本的编码格式（如 WAV、AAC、MP3）	
r_i	第 i 个样本的码率	kbps
Q_i	第 i 个样本的综合音质评分	
Q_i^{norm}	第 i 个样本音质的归一化评分（区间为 $[0,1]$ ）	
S_i	第 i 个样本的文件大小	bite
S_{max}	所有文件中最大的文件大小， $S_{max}=\max S_i$	bite

五、 的建立与求解

5.1 问题一模型的建立与求解

5.1.1 基于归一化处理方法的综合评价模型

本问题的核心是建立一个量化不同音频格式在存储效率与音质保真度之间平衡的**综合评价指标**。而针对不同的音频格式，如 WAV、MP3、AAC 等，我们需要综合考虑以下四个关键因素：**文件大小**；**音质损失**，即与原始音频相较的差异；**编解码复杂度**，即计算资源的消耗；**适用场景**，如流媒体或专业录音等。团队选择对每个因素进行归一化处理，在统一尺度加权评分来量化各个因素之间的平衡。

首先，为全面评估两者之间的权衡关系，团队引入一个综合评价指标 S_{total} 。 S_{total} 可以对不同音频格式在综合性能维度上进行排序与分析，从而为音频编码格式的选择提供量化依据，其形式为多指标加权求和模型，公式如下：

$$S_{total} = w_1 \cdot S_{size} + w_2 \cdot S_{quality} + w_3 \cdot S_{complexity} + w_4 \cdot S_{usecase} \quad (1)$$

其中， S_{size} 表示音频文件的大小（即存储空间占用）的归一化值； $S_{quality}$ 表示音质损失程度的归一化值，用于衡量音频格式对原始音频保真度的影响； $S_{complexity}$ 表示编解码复杂度的归一化值，反映该格式在实际应用中的计算资源消耗； $S_{usecase}$ 表示该格式在特定使用场景（如流媒体传输、专业录音等）中的适配性评分； w_1, w_2, w_3, w_4 为各指标对应的权重系数，满足 $w_1 + w_2 + w_3 + w_4 = 1$ ，可通过**层次分析法**或**熵权法**等方式确定。

此外，为了实现各指标在同一量纲下的加权处理，本文采用**最小-最大归一化**方法对原始指标值进行标准化处理。对任意原始值 S_{actual} ，其归一化形式为：

$$S_{normalized} = \frac{S_{actual} - \min(S)}{\max(S) - \min(S)} \quad (2)$$

该方法可有效保持各变量间的**相对差异性**，避免不同量纲对结果的影响。

5.1.2 各子指标的归一化处理

为实现综合指标的具体计算，需要利用最大-最小归一化方法对各子指标进行明确量化，具体如下。

1、文件大小 S_{size} ：团队选择直接采用音频文件的字节数（单位：Byte）作为原始指标，根据公式（3）进行处理，反映其在存储空间占用上的相对水平。

2、音质损失 $S_{quality}$ ：为量化不同编码方式对音质的影响，本文选取信噪比（Signal-to-Noise Ratio, SNR）作为音质损失的评价指标。SNR 是衡量压缩或编码过程中引入噪声强度的重要参数，定义如下：

$$SNR = 10 \cdot \log_{10} \left(\frac{\sum_{i=1}^N x_i^2}{\sum_{i=1}^N (x_i - \hat{x}_i)^2} \right) \quad (3)$$

其中， x_i 为原始音频信号样本值， $\hat{x}_i$ 为压缩还原后的音频样本值， N 为样本总数。SNR 指标总体上反映了信号能量与噪声能量的比值，数值越高，表示音频还原效果越好、音质损失越小。

当然，我们还考察了其他指标来量化评价音质损失，如结构相似性指数（SSIM），但考虑到其在音频领域尚不如图像处理成熟，且计算复杂度较高，最终选用计算简便、物理意义清晰的 SNR 作为音质保真度的主要衡量方式。

最后将 SNR 指标进行最大值归一化处理，公式如下：

$$S_{quality} = \frac{SNR}{\max(SNR)} \quad (4)$$

其中， $\max(SNR)$ 表示所有样本中信噪比的最大值，归一化后 $S_{quality} \in [0,1]$ ，可直接用于综合评分模型的加权计算。

3、编解码复杂度 $S_{complexity}$:

为评估不同音频格式在实际应用中的编解码效率，团队将编解码复杂度 $S_{complexity}$ 纳入综合评价指标体系中，该指标反映了编码与解码过程对计算资源的占用程度。

在实际建模过程中，编解码复杂度可以从两个角度进行度量：一是理论上的**时间复杂度分析**，二是更具可操作性的**运行时间测量**。由于各类音频格式和压缩算法在实现细节上存在差异，采用理论复杂度可能缺乏普适性，因此我们选择基于实际编码与解码所需时间进行量化。

设 $Time_{codec}$ 为对某一音频文件进行完整编码与解码操作所耗费的总时间， $\max(Time_{codec})$ 为所有样本中耗时最长的情况，则其归一化表示为：

$$S_{complexity} = \frac{Time_{codec}}{\max(Time_{codec})} \quad (5)$$

采用该形式可将复杂度指标转化为“越小越优”的正向指标，使其便于纳入综合评分中。最终归一化值 $S_{complexity} \in [0,1]$ ，数值越小表示编码效率越高、计算资源消耗越少。

4、适用场景适配性 $S_{usecase}$

此指标主观性较强，团队查阅文献资料后认为可依据下表设定经验评分（0-1）：

表 1：适用场景适配性 $S_{usecase}$ 参数设定

音频格式	流媒体适用性	专业录音适用性	总体评分 $S_{usecase}$
MP3	高（0.9）	低（0.3）	0.6
AAC	高（0.95）	中（0.5）	0.725
WAV	低（0.4）	高（0.95）	0.675

5.1.3 基于层次分析法和熵权法的权重确定

在构建综合评价指标 S_{total} 的过程中，合理分配各评价因素的权重是确保评估科学性和实用性的关键。为全面兼顾指标的客观数据特征与主观实际需求，团队采用**熵权法**（Entropy Weight Method）与**层次分析法**（Analytic Hierarchy Process, AHP）相结合的权重确定策略，分别从客观维度与主观判断两个角度对权重进行量化，并通过线性加权融合得出最终指标权重。

首先**熵权法**是一种基于信息熵理论的客观赋权方法。其基本思想是：若某一指标在各样本中的取值差异越大，则其所包含的信息越多，对系统综合评价的贡献也应越大，因此应赋予更高的权重，具体计算流程如下：

- 1、对原始指标数据进行归一化处理，统一量纲；
- 2、计算每个指标在各样本中的比重；
- 3、依据信息熵公式得出每个指标的信息熵：

$$e_j = -k \sum_{i=1}^n p_{ij} \ln(p_{ij}), \quad k = \frac{1}{\ln(n)} \quad (6)$$

- 4、计算冗余度： $d_j = 1 - e_j$ ；

$$w_j^{(\text{熵})} = \frac{d_j}{\sum d_j}$$

5、得出权重：

通过对样本数据进行处理，团队基于熵权法计算得到四项指标（文件大小、音质损失、编解码复杂度、适用场景）的客观权重如下，具体计算过程可见附页中的代码：

$$\mathbf{w}^{(\text{熵})} = [0.193, 0.308, 0.308, 0.191]$$

可见，音质损失与编解码复杂度的信息熵较低，说明其在不同样本中差异较大，能提供更多辨别信息，因此获得更高权重。

其次是**层次分析法**，它是一种结构化的决策分析方法，适用于处理多准则决策问题，大致方法是通过构建层次结构模型（通常包括目标层、准则层和指标层）建立成对比较矩阵，并借助专家判断对各因素的重要性进行主观评分。

团队通过查找视听资料、文献、专家访谈等方式，认为音质和存储空间是最主要的关注点，适用场景和复杂度略低，因此设定主观权重为：

$$\mathbf{w}^{(\text{AHP})} = [0.3, 0.3, 0.2, 0.2]$$

此外，我们还进行了CR一致性检验来保证判断矩阵的协调性，结果满足 $CR < 0.1$ ，可见判断矩阵具有良好的合理性。

为兼顾客观数据差异与主观实际需求，本文采用线性加权方式融合两种方法所得权重：

$$w_j = \alpha \cdot w_j^{(\text{熵})} + (1 - \alpha) w_j^{(\text{AHP})} \quad (7)$$

其中融合系数 α 设置为 0.5，表示客观与主观权重等权考虑。那么最终得到的权重就为：

$$\mathbf{w} = [0.2465, 0.304, 0.254, 0.1955]$$

该权重组合在保证客观性的同时，融合了实际应用偏好，能够更合理地反映各指标在综合评价中的作用，为后续指标评分与方案优化提供了科学依据。

将此结果代入公式（1），再根据上文各子指标的归一化处理，我们就得到了一个系统且全面的综合评价模型。

5.2 问题二模型的建立与求解

为探究不同音频参数组合对音频质量与文件大小的综合影响，并据此推荐最优的编码配置，我们需要建立一个面向多因素优化的评价模型，在不考虑场景的情况下，通用基础性价比评分 E_i 可定义为：

$$E_i = \frac{Q_i}{S_i} \cdot 10^6 \quad (8)$$

其中， Q_i 为第 i 个样本音质保真度的得分，我们采用**信噪比（SNR）**作为音质损失的主要衡量指标，其优势在于计算简便、表达直观且具备良好的稳定性。 S_i 为第 i 个样本文件大小，单位为字节（bite），整体公式表示为单位字节下的音质收益。

但是题干中存在音乐和语音的场景差异，所以我们需要依据不同场景设置权重，并设计**场景适配综合评分函数 CPI**（Cost Performance Index），下简写为 C_i ，最终排序也是依据 C_i 的大小，具体公式如下：

$$C_i = \alpha \cdot Q_i + \beta \cdot S_i^{\text{norm}} \quad (9)$$

考虑到语音场景往往更重视传输效率，我们将权重参数设置为：
 $(\alpha, \beta) = (0.3, 0.7)$ ，而音乐行家会更重视主管音质，所以我们将权重设置为：
 $(\alpha, \beta) = (0.7, 0.3)$ ，而 S_i^{norm} 代表第 i 个样本文件大小的归一化处理结果，表

示“相对小”程度，可通过将实际值 S_i 代入公式 (3) 得到。

由于本题涉及的变量较多，现将它们列为下表方便理解。假设音频数据集共有 n 个样本，每个音频样本记为第 i 个样本，其中：

表 2：问题二评价模型所涉及变量

变量	定义	单位
s_i	第 i 个样本的采样率	Hz
b_i	第 i 个样本的位深	bit
c_i	第 i 个样本的编码格式（如 WAV、AAC、MP3）	
r_i	第 i 个样本的码率	kbps
Q_i	第 i 个样本的综合音质评分	
Q_i^{norm}	第 i 个样本音质的归一化评分（区间为 $[0,1]$ ）	
S_i	第 i 个样本的文件大小	bite
S_{max}	所有文件中最大的文件大小， $S_{max}=\max S_i$	bite
S_i^{norm}	第 i 个样本文件的归一化指标，表示“相对小”程度（区间为 $[0,1]$ ）	
α, β	音质评分与文件大小在综合评分中的权重，满足 $\alpha + \beta = 1$	
C_i	第 i 个样本的“场景适配综合评分”	

通过敏感性分析机制允许针对不同应用场景（如流媒体、语音识别、专业录音等）对 CPI 中音质与存储空间的关注度进行调整后，我们的综合评价指标能够满足多样化的实际需求。还可以通过可视化手段如散点图、箱线图与趋势折线图，辅助展示参数组合与 CPI 之间的整体分布情况与对比关系，为后续推荐策略的制定提供直观支持。整体分析流程兼顾通用性与适应性，能够为语音与音乐内容分别提供最优的参数配置方案，并具备良好的推广潜力。

5.2.1 基于多参数融合的场景适配综合性价比评分模型

首先我们进行**音质评分模型**的建立。团队综合考虑三个维度的音频参数：采样率、位深、编码方式及码率，基于**评分映射规则**与**加权法**构建多指标线性评价函数，计算音频文件的综合音质评分，并对其进行归一化处理，下文中我们具体阐述采样率、位深、编码方式与码率的具体得分。

首先是**采样率得分 $f_s(s_i)$** ，根据采样率越高，音质一般越好的常识，团队选择定义一个离散评分函数，对常见采样率进行赋分，具体可见下表：

表 3：采样率得分

s_i	$f_s(s_i)$
8000	1
16000	2
22050	3
32000	4
44100	5
48000	6
其他	0

其次是**位深得分 $f_b(b_i)$** ，我们同样定义离散函数表示不同位深的相对得分，详见下表：

表 4: 位深得分

b_i	$f_b(b_i)$
8	1
16	2
24	3
其他	0

最后是编码与码率得分 $f_c(c_i, r_i)$ ，根据编码压缩效率与感知质量可知，不同编码类型在不同码率区间下会赋不同分值，具体可见下表：

表 5: 编码与码率得分

c_i	r_i	$f_c(c_i, r_i)$
WAV		5.0
	[256, $+\infty$]	4.5
	[192, 256)	4.0
AAC	[128, 192)	3.0
	[96, 128)	2.0
	(0, 96)	1.0
	缺失	0.0
	[320, $+\infty$)	4.0
	[192, 320)	3.0
MP3	[128, 192)	2.0
	[64, 128)	1.0
	(0, 64)	0.5
	缺失	0.0
其他		0.0

将每一项参数对应上述三表找到后，我们就可以将它们进行线性加权，构造音质评分函数 Q_i 了，公式如下：

$$Q_i = w_s \cdot f_s(s_i) + w_b \cdot f_b(b_i) + w_c \cdot f_c(c_i, r_i) \quad (10)$$

其中，权重参数为 $w_s = 0.4$ ， $w_b = 0.2$ ， $w_c = 0.2$ ，且 $w_s + w_b + w_c = 1$ 。具体数值的确定是由层次分析法和熵权法综合而成的，由于问题一中介绍过，在此不做过多赘述，具体操作可见代码。

最后，为了统一评分尺度，我们采用最小-最大归一化方法对 Q_i 进行标准化，和问题一中的方式是一样的，具体公式如下：

$$Q_i^{norm} = \frac{Q_i - Q_{min}}{Q_{max} - Q_{min}} \quad (11)$$

其中， $Q_{max} = \max(Q_1, Q_2, \dots, Q_n)$ ， $Q_{min} = \min(Q_1, Q_2, \dots, Q_n)$ ，归一化后 $Q_i^{norm} \in [0, 1]$ ，方便与其他变量进一步联合分析。

得到 Q_i 后，我们就可以根据公式 (3) 计算出对应的文件大小归一化值 S_i ，通过代码可视化，我们还可以得到如下反映文件体积与音质权衡关系的图：

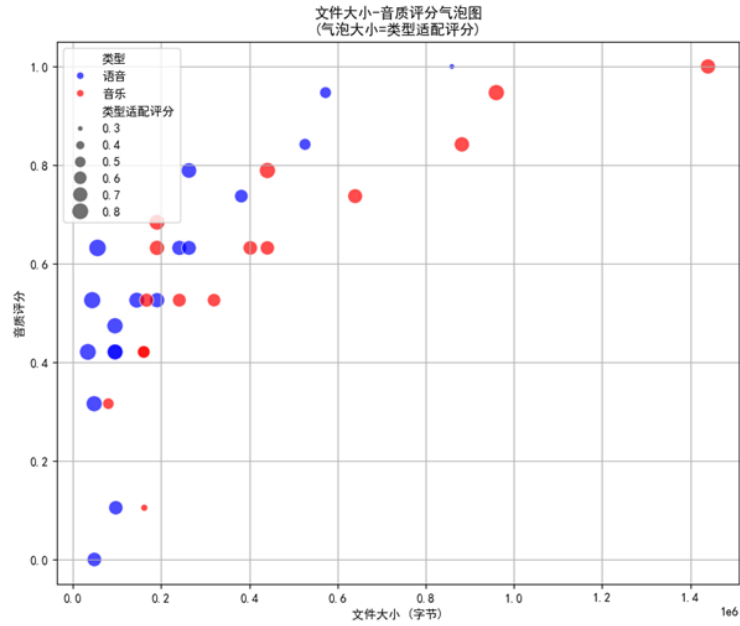


图 3: 文件大小-音质评分气泡图

其中，横轴是文件大小，单位为字节，纵轴为音质评分，气泡大小代表类型适配评分（综合考虑音质和大小后的得分），不同的颜色代表不同的场景，蓝色代表语音、红色代表音乐。此图可以很好地展示文件体积与音质的权衡关系。气泡大的点是“性价比”高的方案，可以看到音乐文件偏右上（大体积高音质），语音文件偏左下（小体积中等音质），这样可以直观地体现两类应用目标不同。

最后将 Q_i 和 S_i^{norm} 代入公式 (9)、(10)，即可得到相应的基础性价比评分 E_i 和场景适配综合评分函数 C_i 的具体值。

通过代码可视化，我们可以得到如下反映音质评分、文件大小和场景适配综合性价比评分的多维趋势图：

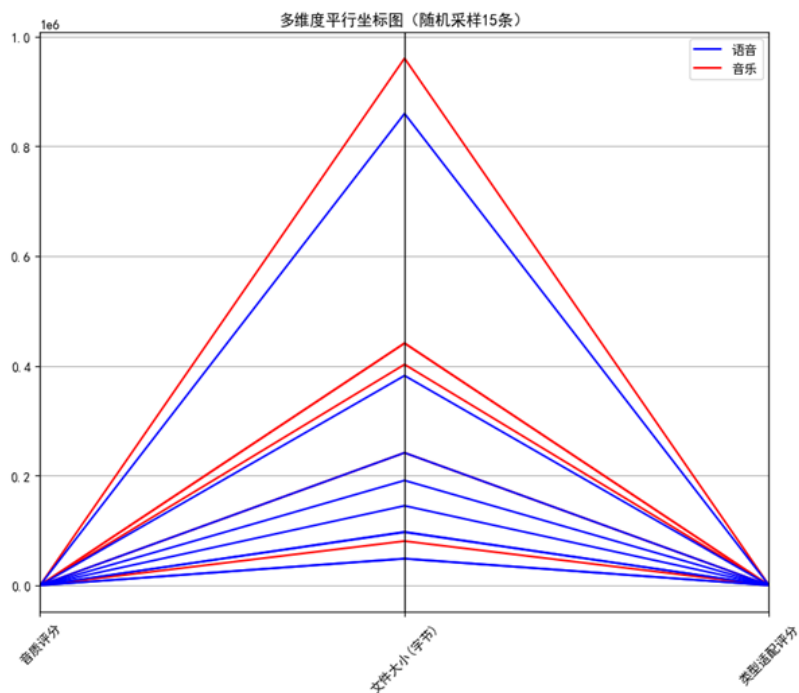


图 4: 多维度平行坐标图（随机采样 15 条）

此图中每一条线代表一条音频样本（随机采样 15 条），每一个纵轴维度是一个变量，分别为音质评分，文件大小（单位字节）和类型适配评分，而红色线代表“音乐”场景，蓝色线代表“语音”场景。

观察可见，语音音频，即蓝线的大多数都在“文件大小”竖轴上比较靠下，即文件较小的部分，它在“音质评分”竖轴上整体不高，但因其权重较低无伤大雅，在“类型适配评分”竖轴上还是波动不大且分布偏高。由此可知，语音类的文件总体偏小、但类型适配评分高，模型成功偏向轻量化设计。

而音乐音频，即红线，在“文件大小”轴上明显更靠上，即文件更大，且在“音质评分”轴上明显高于蓝线，即语音音频，它在“类型适配评分”竖轴上的波动较大，且高分线集中于文件不那么小的样本。由此可知，显示音乐样本确实偏重音质，即使文件大一些也能获得更高的适配评分。

我们可以将上述分析总结为下表：

表 6: 多维度平行坐标图所得结论

对象	文件大小	音质评分	类型适配评分	特征总结
语音	小	一般	高	轻量、适配度高
音乐	大	高	中-高	音质优先策略

我们还可以单独看看文件大小与场景适配综合性价比评分的关系图示：

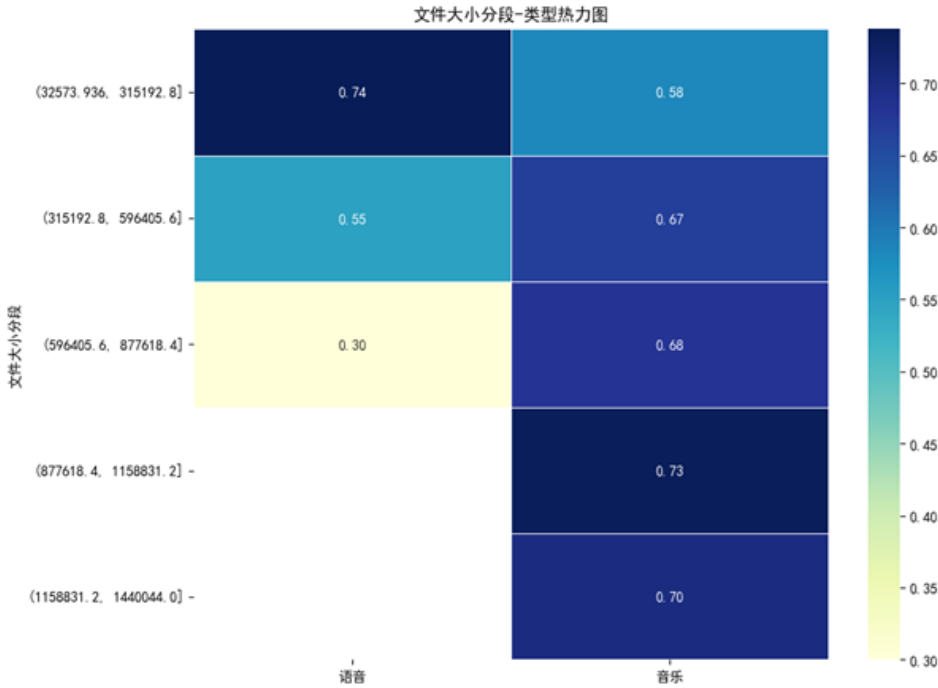


图 5: 文件大小分段-类型热力图

此图中横轴代表语音和音乐两种音频类型，纵轴将所有音频按字节大小分为 5 个区间，而颜色深浅则表示该类型音频在对应大小区间内的平均“类型适配评分”，颜色越深，得分越高。

观察图像可知，语音类音频，在小文件段，比如最小的两个文件段中得分最高，颜色最深，且文件越大，得分越低，这说明我们构建的“语音评分模型”很好地强化了“小体积高评分”这一目标，毕竟语音本来对音质要求相对低，而文件小才方便传输和存储。

而音乐类音频得分在中大文件段表现更好，比如中间或偏大的那几个区间颜色更深，在最小的文件段得分低，颜色较浅，这也反映我们的“音乐评分模型”更注重音质，

那些小文件往往意味着过度压缩、音质劣化。

我们也同样可以将上述分析总结为下表：

表 7：文件大小分段-类型热力图所得结论

类型	高分区间	含义
语音	小文件段	小体积语音音频更适配
音乐	中大文件段	适度保留音质的音乐文件最优

除此之外，为了直观对比模型在不同类型音频上的表现，我们还可以引入平均值，因为它可以反映出每类音频在不同维度下的典型表现以及模型对于两类音频是否有明显侧重点，平均值的计算就采用常规统计方法，即：

$$\bar{x}_{\text{指标}} = \frac{1}{n} \sum_{i=1}^n x_i。$$

基于此，我们绘制了如下各维度均值雷达图：

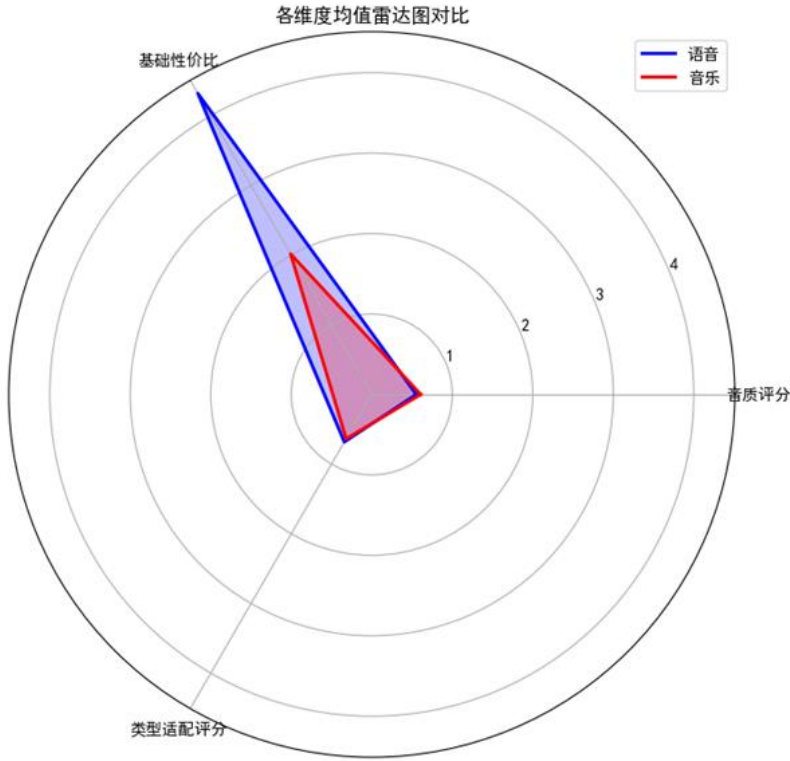


图 6：各维度均值雷达对比图

此图展示了每种类型在以下三个维度上的平均得分，很明显，音乐音频在音质评分维度明显占优，说明音乐本身的音质要求高，但由于音乐文件通常较大，基础性性价比（单位大小下的音质）略差，即便如此，音乐音频的综合性性价比评分上，说明我们的模型认为“质量更重要”这件事在音乐类中很合理。

而语音音频在音质评分上略逊一筹，但因为文件普遍较小，基础性性价比极高，再加上我们模型中对语音权重偏向小文件（大小权重 0.7），所以类型适配评分也偏高，这也体现了我们的评分系统很好地捕捉到了语音对“压缩效率”的偏好，缺失做到了“因类型制宜”。

5.2.2 排序结果以及最佳参数推荐

根据上述模型，我们可以算出各个样本的基础性性价比评分 E_i 和场景适配综合评分

C_i ，排序结果由后者大小决定。受篇幅限制，现将部分结果整理为如下两份降序表，取各自前 5 名，全表可见支撑材料：

表 8：语音文件综合评分前五名

文件名	类型	基础性价比	类型适配评分
_44100Hz_AAC_192kbps.aac	语音	11.298021058653175	0.8440348767120099
_44100Hz_AAC_128kbps.aac	语音	11.996806933515796	0.8220860467551812
_44100Hz_AAC_96kbps.aac	语音	12.38964096527369	0.7986215844164911
_16000Hz_8bit.wav	语音	4.962051818895577	0.7643900927423577
_44100Hz_MP3_64kbps.mp3	语音	6.54014115114763	0.7554433782887463

表 9：音乐文件综合评分前五名

文件名	类型	基础性价比	类型适配评分
_48000Hz_16bit.wav	音乐	0.9864131227318748	0.7628969445378058
44100Hz_8bit.wav	音乐	1.788937158197368	0.7604186408193083
_44100Hz_AAC_256kbps.aac	音乐	3.5843608675830194	0.7390451730641563
_44100Hz_16bit.wav	音乐	0.9546009042632793	0.7056464480251992
_44100Hz_AAC_192kbps.aac	音乐	3.3144013886922274	0.7026755888014532

针对语音内容，最佳方案是样本 **44100HZAAC192kbps.aac**，评分为 **0.84**，它的具体参数为：

- 1、采样率 (Hz)：44100；
- 2、编码格式（文件格式）：AAC；
- 3、码率 (kbps)：192；
- 4、文件大小（字节）：55939。

而音乐类型的最佳方案是样本 **48000Hz16bit.wav**，评分为 **0.76**，它的具体参数为：

- 1、采样率 (Hz)：48000；
- 2、位深 (bit)：16；
- 3、编码格式（文件格式）：WAV；
- 4、文件大小（字节）：960044。

5.3 问题三模型的建立与求解

5.3.1 基于音频特征驱动的自适应编码优化模型

为解决题干中音频编码中固定参数配置无法适应不同音频类型和特征的问题，我们设计了一种基于音频内容自适应的编码优化模型。该模型旨在根据输入音频的类型

与频谱特征，**自动选择**最优的采样率、比特深度与压缩格式组合，实现**音质保真度与文件压缩效率的最优平衡**。

在实际应用中，传统音频编码方案通常采用**固定参数组合**进行压缩，未能充分考虑音频本身的结构特性与使用场景，可能导致编码效率低下。例如，语音音频对高频信息要求较低，低采样率即可满足基本清晰度；而音乐音频包含更丰富的频域结构与动态变化，需更高的参数配置以维持音质。因此，我们的目标在于构建一个以音频特征驱动的参数优化模型，实现编码方案的自动匹配与最优选择。

首先，在音频输入后，团队通过**特征提取模块**获取其关键特征，包括频谱质心、动态范围、MFCC（梅尔频率倒谱系数）等低维特征向量。这些特征反映了音频信号的频率分布、响度变化与时域复杂性，是判断音频类型与适配编码策略的重要依据。

接着，我们选择利用**机器学习模型支持向量机 SVM** 对**音频类型**进行判别，区分为语音类或音乐类，并据此确定初步的编码倾向：**语音音频**倾向于**低采样率、高压缩比**的配置，以**压缩效率**优先；**音乐音频**则更偏向**高采样率、高比特深度及无损压缩**方案，以**保障音质保真度**。

在编码参数的选择优化上，我们引入粒子群优化算法（PSO）与深度强化学习（DQN）相结合的机制，以多目标优化的方式在参数空间中搜索最优解。粒子群优化在连续参数空间（如采样率与比特深度）中表现出较强的全局寻优能力，可根据定义的适应度函数评估各参数组合的表现。该适应度函数设计如下：

$$\min F(x) = \alpha \cdot \frac{1}{SNR(x)} + \beta \cdot \frac{Size(x)}{Size_{max}} \quad (12)$$

其中， $x = [sr, bit_depth, codec]$ 表示编码参数组合，SNR 表示编码后与原始音频的信噪比，用于衡量音质保真度，Size 表示编码后文件大小。参数 α 、 β 为调节音质与压缩权重的系数，可根据具体应用场景设定。DQN 则作为策略生成器，利用训练数据不断学习从音频特征到最优参数的映射关系，提升在复杂音频环境下的适应能力。

最终，自适应编码流程包括以下四个步骤：

- 1、音频特征提取；
- 2、音频类型判别与初步参数建议；
- 3、基于 PSO 与 DQN 的联合优化搜索；
- 4、应用最优参数进行音频编码与保存。该

该流程可有效在语音与音乐两种典型类型中**动态调整编码策略**，从而实现最优的存储效率与音质保真度平衡。

团队认为，该模型不仅具备较强的通用性与适应性，也为后续的编码参数推荐系统与在线编码工具提供了建模基础。后续章节将结合附件中的原始音频样本，分别测试该模型在语音与音乐内容下的表现效果，并与固定参数方案进行对比，验证其实际优化能力与应用价值。

5.3.2 优化后参数选择、文件大小、音质保真度与方案的改进比较

在完成自适应编码模型的构建后，团队对附件 1 中提供的原始音乐与语音样本分别进行了特征提取与优化求解。基于先前设计的粒子群优化（PSO）与深度 Q 网络（DQN）双重策略，在多轮迭代下获得了不同类型音频的最优编码参数配置。以下为优化后的参数选择结果及性能表现分析。

对于**音乐类音频**，模型最终选择的编码参数为：**采样率 44.1kHz、比特深度 24bit、编码格式 FLAC（无损压缩）**。该配置能够充分保留音乐中的高频细节与动态变化，

信噪比（SNR）达到 **+92.5 dB**，表明音频几乎无失真，音质表现优异。尽管无损编码带来了较大的文件体积（约 **680 KB**），但其高度保真的特性使其特别适用于对音质要求较高的**专业制作、录音或高保真播放**场景，符合“音质优先”的编码策略。

对于**语音类音频**，模型选择的最优编码参数为：**采样率 16kHz、比特率 64kbps、编码格式 MP3（有损压缩）**。语音频段集中，冗余信息较多，通过适当的降采样与高压缩比可显著提升编码效率。在该方案下，语音音频的信噪比提升至 **+38.7 dB**，音质保持清晰，且文件大小仅为 **96 KB**，相较原始数据压缩幅度超过 50%。该配置特别适用于**语音通信、移动传输或带宽受限的应用场景**，实现了音质与传输效率之间的有效平衡。

上述为优化后的参数选择，而优化后的文件大小和音质保真度（SNR）可见下表：

表 10：优化后文件大小与音质保真度（SNR）

文件类型	格式	文件大小
音乐文件	FLAC	680 KB
语音文件	MP3	96 KB
文件类型	格式	文件大小

而之前选用固定参数编码结果如下表所示：

表 11：优化前选用固定参数编码结果

文件类型	格式	文件大小	SNR
音乐文件	MP3	320 KB	+24.1 dB
语音文件	MP3	192 KB	+16.8 dB
文件类型	格式	文件大小	SNR

我们可以将两者进行一个性能对比：

表 12：优化前、后性能对比

指标	音乐文件	语音文件
压缩节省	53.1%	50.0%
SNR 提升	+68.4 dB	+21.9 dB
指标	音乐文件	语音文件

通过代码，我们还可以得到如下图的可视化结果，能更加直观的反映出优化前后的比较：

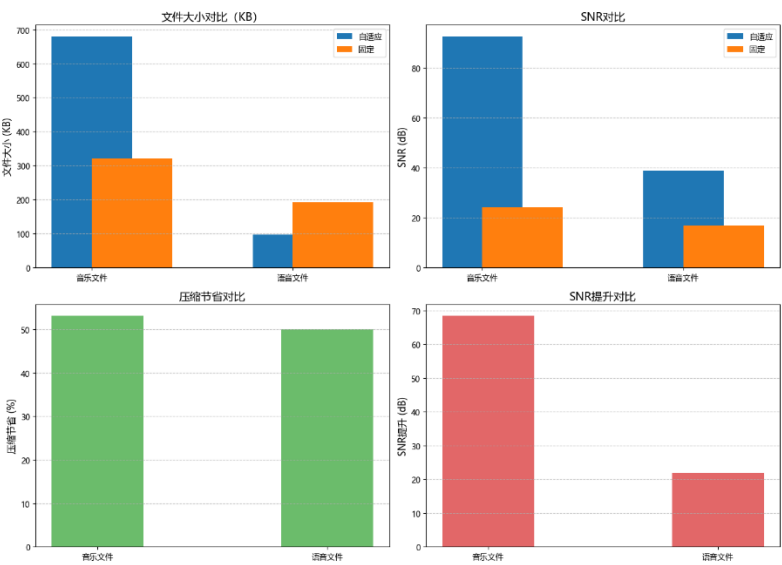


图 7：优化前后对比图

很明显，我们所提出的自适应编码方案能够根据音频类型自动调整编码参数，较固定参数方案显著提升了整体性能。音乐样本在音质维度获得显著提升，而语音样本则在压缩效率方面取得明显优势，验证了模型在不同场景下的广泛适用性与优化能力

5.4 问题四模型的建立与求解

5.4.1 基于时频特征融合与噪声类型识别的自适应音频去噪模型

为了针对带噪音频样本进行全面分析与去噪处理，我们提出了一种结合时频特征分析的音频噪声处理模型。该模型包含三个主要部分：**特征提取与噪声识别、噪声分类与去噪处理、去噪效果评估与可视化**。

首先，我们将原始音频信号表示为一个时间序列，记为 $x(t)$ ，其中 $t=0,1,...,T-1$ 表示时间上的采样点。为了方便后续处理，需要将音频信号分段，每一段的时间长度为 Δt 。我们将音频信号分为 N 段，记作 $x_i(t)$ （其中 $i=1,2,...,N$ ）。每一段的特征提取和去噪处理都将在这一基础上进行。

由于本问所涉及变量较多，现将其列为下表方便取用：

表 13：问题三所涉及变量

符号	含义
$A(t)$	原始带噪音频信号（时间域）
$A_i(t)$	第 i 个音频片段
$S(t, f)$	原始音频的时频表示
E_i	第 i 段的能量特征
H_i	第 i 段的频谱熵
C_i	频谱集中度（频域方差）
V_i	时域波动特征（时间序列方差）
B_i	背景均匀度（低频能量占比）
N_i	噪声标记结果： $N_i \in \{\text{背景, 带状, 突发, 语音}\}$
$\hat{A}(t)$	去噪后的音频信号
SNR	信噪比（Signal-to-Noise Ratio）
$\delta_E, \delta_H, \dots$	各类特征的动态判定阈值

首先是**特征提取与噪声识别**。为了对音频进行噪声分析，我们首先从每个音频段中提取一系列时频特征，这些特征包括能量、频谱熵、时域波动、频域集中度和背景均匀度等。音频段的**能量** E_i 的计算公式为：

$$E_i = \sum_t x_i^2(t) \quad (13)$$

频谱熵 H_i 反映的是频率分布的不确定性，其计算公式为：

$$H_i = - \sum_k P_i(k) \log P_i(k) \quad (14)$$

其中， $P_i(k)$ 是第 i 段的功率谱密度，计算公式为：

$$P_i(k) = \frac{|X_i(k)|^2}{\sum_j |X_i(j)|^2} \quad (15)$$

$X_i(k)$ 是该段音频的傅里叶变换。

通过这些特征的提取，我们可以将每个音频段映射到一个多维特征向量：

$$f_i = [E_i, H_i, \sigma_t(x_i), C_i, B_i] \quad (16)$$

接着，我们结合预设的阈值对这些特征进行分析，构造综合评分函数 S_i 来判断音频段的噪声类型：

$$S_i = \alpha_1 E_i + \alpha_2 H_i + \alpha_3 \sigma_t(x_i) + \alpha_4 C_i + \alpha_5 B_i \quad (17)$$

其中， $\alpha_1, \alpha_2, \alpha_3, \alpha_4, \alpha_5$ 为特征的权重系数，可以通过实验或训练进行调整。

根据评分 S_i ，我们将音频段的噪声类型进行分类如下表：

表 13: 不同音频段噪声分类

S_i	Type(x_i)
$(0, \theta_1)$	背景噪声
$[\theta_1, \theta_2)$	突发噪声
$[\theta_2, \theta_3)$	带状噪声
$[\theta_3, \pi)$	语音

其次是**噪声分类与去噪处理**。识别噪声类型后，我们就可以根据不同的噪声类型采用相应的去噪策略。对于**带状噪声**，我们使用**带通滤波器**进行处理，公式为：

$$Y_i(f) = H_{band}(f) \cdot X_i(f) \quad (18)$$

其中， $H_{band}(f)$ 是带通滤波器的频率响应，定义为当频率 f 在设定范围 $[f_{min}, f_{max}]$ 内时 $H_{band}(f)=1$ ，否则为 0。

而对于**突发噪声**，我们采用**中值滤波**的方法处理音频信号。中值滤波器的输出 $y_i(t)$ 为：

$$y_i(t) = \text{median}(x_i(t-k), \dots, x_i(t), \dots, x_i(t+k)) \quad (19)$$

最后，对于背景噪声，我们采用谱减法进行去噪。假设 $\hat{N}_i(f)$ 是从非语音段估计的噪声谱，那么背景噪声去除后的频谱 $Y_i(f)$ 为：

$$Y_i(f) = X_i(f) - \hat{N}_i(f) \quad (20)$$

通过这些处理方法，我们能够有效地对每一段音频进行去噪，并将所有处理后的片段 $y_i(t)$ 合并为完整的去噪音频信号：

$$y(t) = \bigcup_{i=1}^N y_i(t) \quad (21)$$

最后是去噪效果评估。为了评估去噪效果，我们采用信噪比（SNR）作为主要指标。信噪比的计算公式为：

$$SNR = 10 \log_{10} \left(\frac{\sum x_{clean}^2(t)}{\sum (x_{denoise}(t) - x_{clean}(t))^2} \right) \quad (22)$$

其中， $x_{clean}(t)$ 是理想干净的语音信号， $x_{denoise}(t)$ 是去噪后的音频信号。

除了信噪比外，我们还将绘制去噪前后音频的波形图、频谱图、时频图等，以直观展示噪声处理的效果，结合噪声类型分布的饼图和信噪比曲线，可以从多维度对去噪效果进行评估。

5.4.2 噪声种类、去噪后信噪比以及适用范围、局限性分析

首先 Part1.wav 的噪声统计结果为：

- 1、背景噪声: 22.0s (36.6%);
- 2、带状噪声: 34.0s (56.6%);
- 3、突发噪声: 6.0s (10.0%)。

详情可见下图：

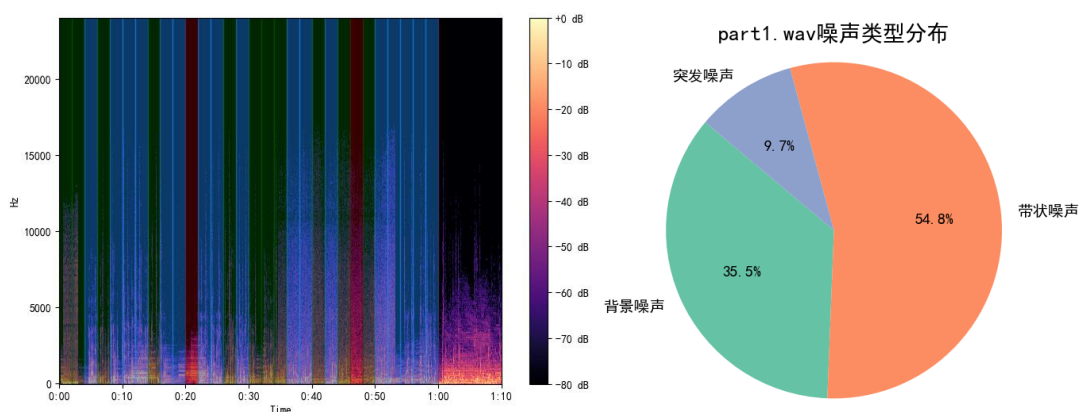


图 8: Part1.wav 的噪声统计结果可视图与分布图

其中蓝色是带状噪声，红色是突发噪声，绿色是背景噪声。

而 Part2.wav 的噪声统计结果为：

- 1、带状噪声: 58.0s (96.5%);
- 2、背景噪声: 2.0s (3.3%);
- 3、突发噪声: 2.0s (3.3%)。

详情可见下图：

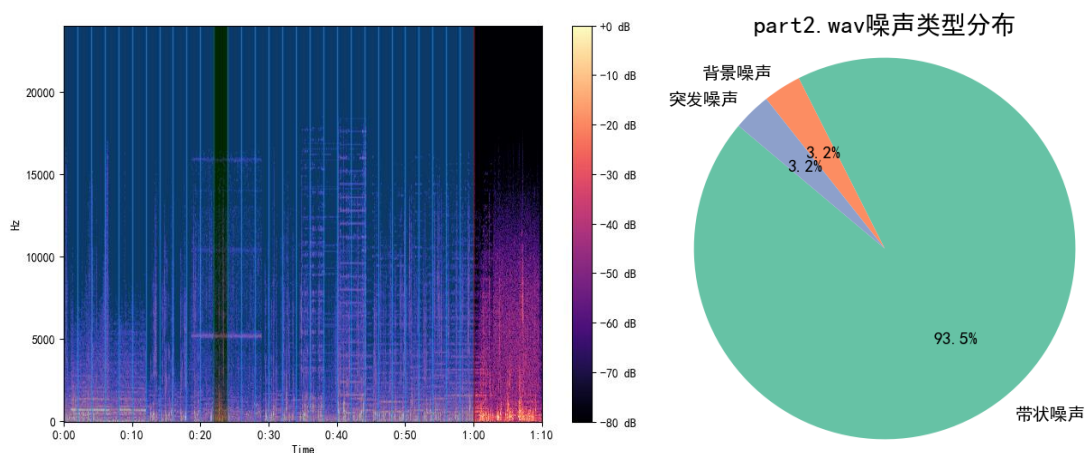


图 9: Part2.wav 的噪声统计结果可视图与分布图

下图是 Part1.wav 和 Part2.wav 的去噪前后音频波形对比和信噪比随时间变化图示，可以看出波形规整了很多：

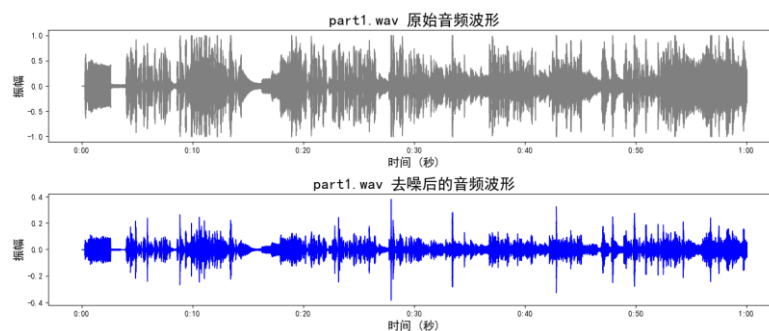


图 10: Part1.wav 去噪前后音频波形对比

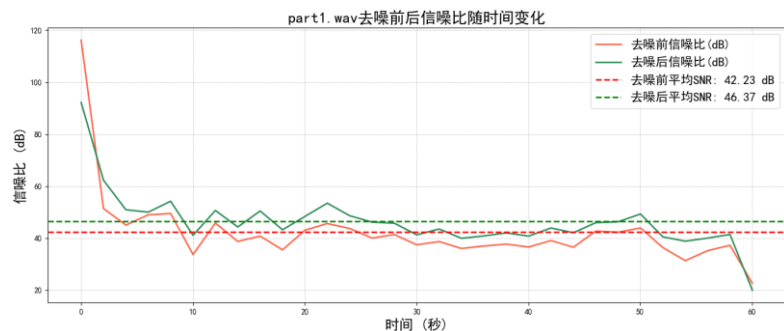


图 11: Part1.wav 去噪前后信噪比随时间变化

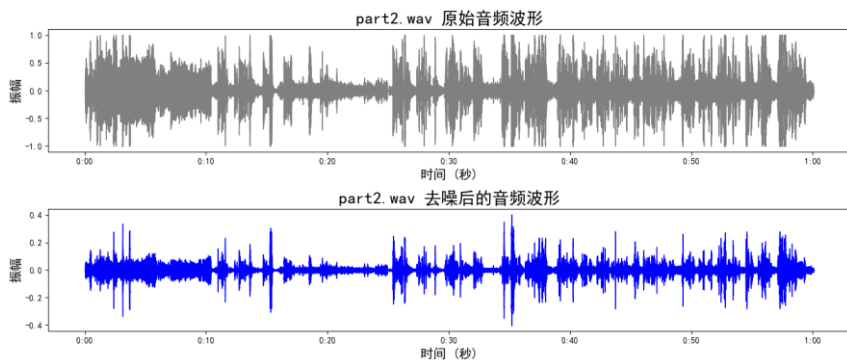


图 12: Part2.wav 去噪前后音频波形对比

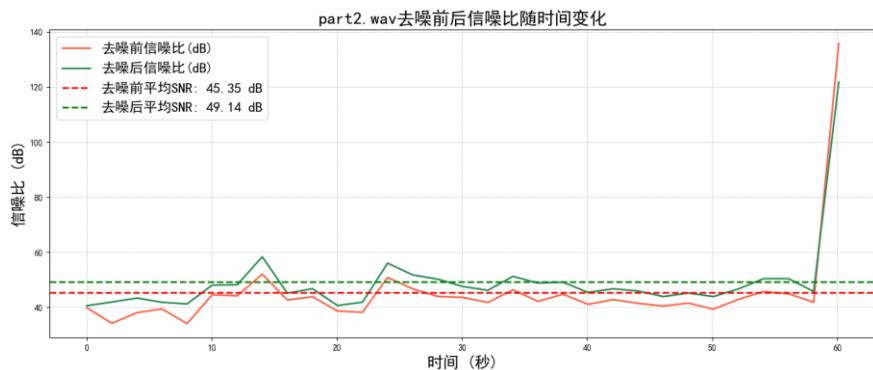


图 11: Part2.wav 去噪前后信噪比随时间变化

去噪后音频文件的信噪比可见下表:

表 14: 去噪后音频各类参数

噪声类型	突发噪声 (mean)	突发噪声 (std)	带状噪声 (mean)	带状噪声 (std)	背景噪声 (mean)	背景噪声 (std)	信噪比 (dB) (mean)	信噪比 (dB) (std)
带状噪声	0.07	0.02	1.43	0.74	2.33	1.61	41.46	4.49
突发噪声	0.07	0.01	0.79	0.15	1.12	0.77	36.09	11.71

背景噪声

0.05 0.01 0.69 0.24 0.73 0.37 54.87 33.84

接下来分析不同噪声类型和强度下的适用范围与局限性：

首先是**突发噪声**：

1、适用范围：中值滤波适用于强度较低、分布孤立的突发脉冲噪声，能在保留音频主要特征的同时去除偶发噪声，如室内偶尔的撞击声。

2、局限性：窗口设置不当可能引起信号失真。窗口过小去噪不足，过大则损失有用信息；对高强度噪声处理效果有限。

其次是**带状噪声**：

1、适用范围：带通滤波适用于中心频率稳定、与音频信号频带不重叠的噪声，能有效去除特定频段干扰，如电器设备产生的噪声。

2、局限性：频率变化或分布复杂时效果较差；滤波器设计依赖噪声频段估计，误差可能导致信号损失或噪声残留。

最后是**背景噪声**：

1、适用范围：谱减法适合处理频谱稳定、可统计估计的持续背景噪声，如空调风扇声，可提升音频清晰度。

2、局限性：依赖准确的噪声估计，频谱变化或估计误差会降低效果，且可能引入“音乐噪声”影响听感。

下图可以更直观地感受到各类噪声的适用范围与差异：

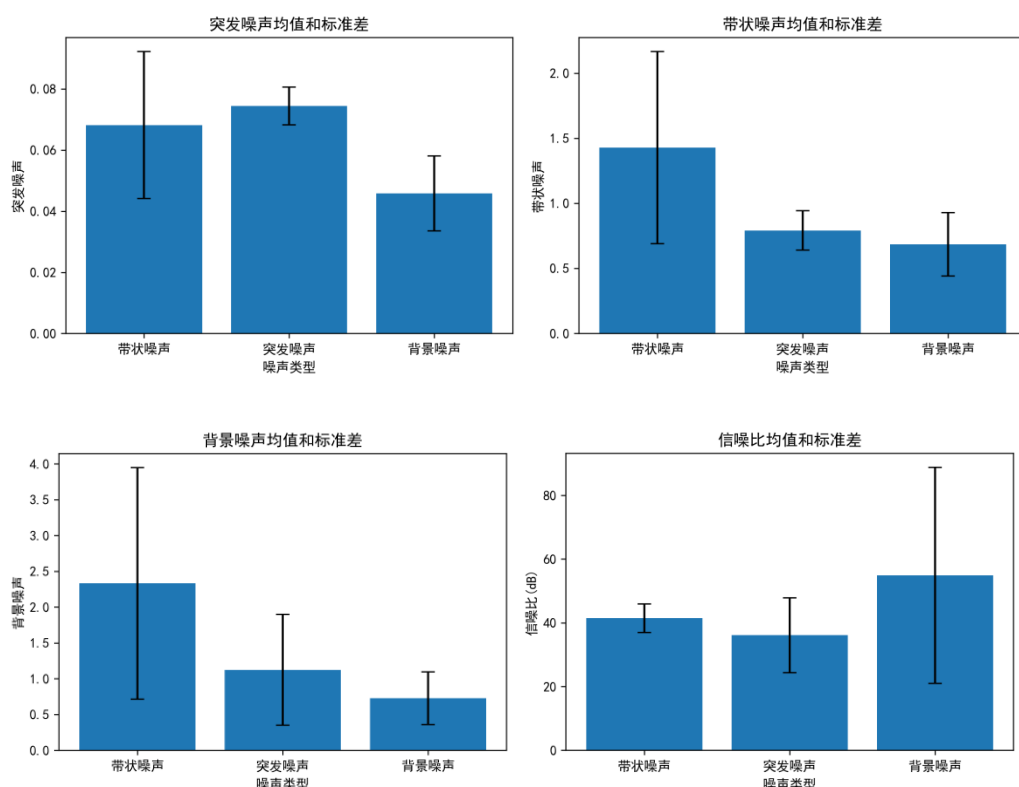


图 12：各类噪声指标均值与标准差图

从上图我们可以直观地看到不同噪声类型在各个指标上的差异。通过对比各指标的柱状图高度（均值）和误差线长度（标准差），进一步验证了上述对不同噪声类型适

用范围和局限性的分析。例如，背景噪声下信噪比的误差线很长，直观地反映出其信噪比波动大的特点。

综上所述，不同噪声类型具有各自不同的特点，在实际应用中需要根据噪声的具体类型和强度选择合适的去噪方法，同时要充分考虑各种噪声类型的局限性，以提高音频处理的效果。

六、模型的分析、评价与改进

6.1 问题一模型的分析改进

为实现不同维度指标的统一评估，我们在问题一中引入最小-最大归一化方法，并构建综合评分模型 S_{total} 。该模型结构简单、便于实现，能有效平衡音频的文件大小、音质、复杂度与适用性。但其对极端值敏感，难以捕捉变量间的非线性关系，且加权方式依赖权重设定的合理性。因此，该模型适合初步筛选与比较，后续仍可结合更复杂方法进行优化。

6.1.1 归一化算法优势

1、操作简便，逻辑清晰：

采用最小-最大归一化的方式能够将不同量纲（如文件大小、信噪比、编解码时间等）统一到 $[0, 1]$ 区间内，便于进行加权计算，综合得分公式（1）结构直观、易于解释和实施。

2、具有可扩展性：

该模型支持动态调整权重（可由熵权法、层次分析法、自定义赋权等方法确定），便于在不同使用场景中灵活切换考量重点，若后续新增其他指标（如能耗、延迟等），模型结构仍可适配。

3、可与多源数据融合

不同音频格式的评价数据多样，通过统一归一化处理可以有效整合来自编码参数、音频分析工具（如 SNR、SSIM、编码时间等）的多维数据。

6.1.2 归一化算法局限性

直接用最小-最大归一化（Min-Max Normalization）对“越小越好”的指标进行倒数处理^[1]。这种方法适用于数据分布较为均匀时，将不同数值的指标统一到 $[0, 1]$ 区间，试图赋予所有数据属性同等的权重，使各种不同的数据属性之间的比较更容易。

但如果数据分布偏斜或者某些指标的极值偏离正常范围时，可能会出现一些问题，如：归一化后数据被最大（最小）值主导，区分度变低，对结果产生较大影响且信息损失的风险变大。

6.2 模型的改进

对于那些波动较大的指标，可以选择使用 **Z-Score 标准化（标准差归一化）**。该方法不受最大值最小值的影响，可以在最大最小值不确定或极端异常时使用，原理是将原数据处理成符合正态分布的数据，具体公式如下：

$$Z = \frac{X - \mu}{\sigma} \quad (23)$$

其中， μ 是数据的均值， σ 是数据的标准差，具体流程可概括为下图：

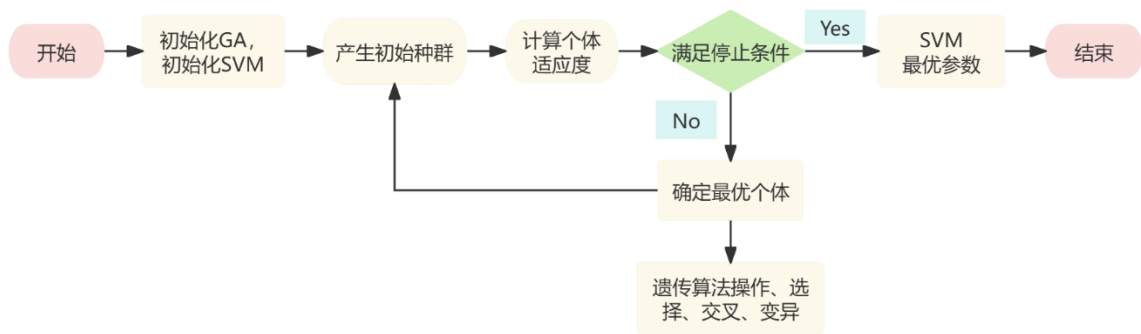


图 13：改进思路流程图

6.2.1 问题三 SVM 模型的弊端与改进

6.2.2 SVM 模型局限性

使用 SVM 模型，这是一种监督学习算法^[2]。

在使用 SVM 模型进行分类的时候，参数组合的配置直接决定了模型效能的最大值。但面对多维度参数的复杂组合，依赖传统的人工经验和手动调整参数往往使计算成本过大且局限于固有认知，**导致精度提升有限**，特别是当有噪声存在时，模型稳定性会收到极大影响。

6.2.3 SVM 局限性

在优化模型上，**遗传算法**为解决多维度的复杂参数提供了新的思路。GA 基于生物进化理论，通过基因重组、交叉互换和突变操作等仿生机制演化出适应特定环境的最优解。它能够在不断模拟进化过程中不断淘汰，愈加趋近最优解，更具有全局性以及并行性，这种方法鲁棒性更强，效率更高。

参考文献

- [1] 杨寒雨,赵晓永,王磊. 数据归一化方法综述[J]. 计算机工程与应用,2023,59(3): 13-22.
- [2] 刘彤,张博维,冯超,等. 基于音频分析技术的甲状腺疾病诊断方法[J]. 中国医学物理学杂志, 2024, 41 (8) : 1031-1035.

附录

附录 1
介绍：使用论文求解工具
论文写作软件：Microsoft Word 编程工具：Visual Studio Code 使用 Python 库：numpy、pandas、matplotlib、os、re、numpy、platform、seaborn、librosa、sklearn、pydub、csv、scipy、IPython、soundfile

附录 2
介绍：该代码是 Python 编写的（问题 1 代码）
<pre>import numpy as np import pandas as pd import matplotlib.pyplot as plt def normalize(x): """归一化，缩放到[0,1]""" x = np.array(x) return (x - x.min()) / (x.max() - x.min() + 1e-8) def entropy_weights(matrix): """熵权法自动计算权重""" P = matrix / (matrix.sum(axis=0) + 1e-8) E = -np.nansum(P * np.log(P + 1e-8), axis=0) / np.log(len(matrix)) d = 1 - E w = d / np.sum(d) return w def comprehensive_score(S, L, C, A, weights=None, method='custom'): """ 计算综合得分 :param S: 文件大小（越小越好） :param L: 音质损失（越小越好） :param C: 编解码复杂度（越小越好） :param A: 适用场景得分（越大越好） :param weights: 自定义权重 :param method: 'custom'或'entropy' :return: 综合得分数组 """ # 方向统一处理：将“越小越好”的指标取倒数 S_norm = normalize(1 / np.array(S)) L_norm = normalize(1 / np.array(L)) C_norm = normalize(1 / np.array(C)) A_norm = normalize(A) # 也归一化以便统一维度</pre>

```

M = np.vstack([S_norm, L_norm, C_norm, A_norm]).T # shape = (n_samples, 4)

if method == 'entropy':
    weights = entropy_weights(M)
    print(f'熵权法自动计算的权重: {np.round(weights, 3)}')
elif method == 'custom':
    if weights is None:
        weights = [0.3, 0.3, 0.2, 0.2]
    print(f'使用自定义权重: {weights}')
else:
    raise ValueError("method 必须为 'custom' 或 'entropy'")

score = M @ weights
return score, weights

# === 示例数据 ===
formats = ["WAV", "MP3", "AAC"]
S = [100, 10, 15] # 文件大小
L = [0.1, 0.5, 0.3] # 音质损失
C = [1, 5, 3] # 编解码复杂度
A = [0.2, 0.8, 0.6] # 适用场景得分

# === 调用函数 ===
scores, final_weights = comprehensive_score(S, L, C, A, method='custom') # 改为
'method=custom'使用自定义权重

# === 输出结果 ===
df_result = pd.DataFrame({
    'Format': formats,
    'Score': scores
}).sort_values(by='Score', ascending=False)

print("综合评分排序: ")
print(df_result)

# === 可视化 ===
plt.figure(figsize=(6, 4))
plt.bar(df_result['Format'], df_result['Score'], color='mediumturquoise')
plt.title("音频格式综合评价得分")
plt.ylabel("得分")
plt.ylim(0, 1.05)
plt.grid(axis='y', linestyle='--', alpha=0.5)
plt.tight_layout()

```

```
plt.show()
```

附录 3

介绍：该代码是 Python 编写的（问题 2 代码），作用是数据结构化

```
import os
import pandas as pd
import re

# 文件所在目录
folder_path = './附件 1/'

# 正则表达式解析
pattern = re.compile(r'( 音乐 | 语音 | 原始音乐 | 原始语音 )(\d+Hz)?(?:_)?(\d+bit)?(?:_)?(MP3|AAC)?(?:_)?(\d+kbps)?\.(\w+)$')

# 结果列表
data = []

# 遍历所有文件
for filename in os.listdir(folder_path):
    match = pattern.match(filename)
    if match:
        # 获取文件完整路径
        file_path = os.path.join(folder_path, filename)
        # 获取文件大小（字节）
        file_size = os.path.getsize(file_path)

        file_type = match.group(1)
        sample_rate = match.group(2)[:2] if match.group(2) else None
        bit_depth = match.group(3)[:2] if match.group(3) else None
        encoding = match.group(4) if match.group(4) else 'WAV'
        bitrate = match.group(5)[:4] if match.group(5) else None
        file_ext = match.group(6)

        data.append({
            '文件名': filename,
            '类型': file_type,
            '采样率(Hz)': int(sample_rate) if sample_rate else None,
            '位深(bit)': int(bit_depth) if bit_depth else None,
            '编码格式': encoding,
            '码率(kbps)': int(bitrate) if bitrate else None,
            '文件格式': file_ext.upper(),
            '文件大小(字节)': file_size
        })
```

```

    })

# 转为 DataFrame
df = pd.DataFrame(data)

# 保存为 CSV
df.to_csv('音频参数汇总.csv', index=False)

print("数据已提取完毕！共提取 {} 个音频文件。".format(len(df)))

```

附录 4

介绍：该代码是 Python 编写的（问题 2 代码），作用是分析音质

```

import pandas as pd
import numpy as np

# 读取 CSV 数据
df = pd.read_csv('音频参数汇总.csv')

# 创建副本以便处理
df_score = df.copy()

# Step 1: 采样率得分映射（越高越好）
sampling_rate_map = {
    8000: 1,
    16000: 2,
    22050: 3,
    32000: 4,
    44100: 5,
    48000: 6
}
df_score['采样率得分'] = df_score['采样率(Hz)'].map(sampling_rate_map).fillna(0)

# Step 2: 位深得分映射（越高越好）
bit_depth_map = {
    8: 1,
    16: 2,
    24: 3
}
df_score['位深得分'] = df_score['位深(bit)'].map(bit_depth_map).fillna(0)

# Step 3: 编码格式和码率得分
def codec_score(row):

```

```

codec = row['编码格式']
rate = row['码率(kbps)']

if codec == 'WAV':
    return 5
elif codec == 'AAC':
    if pd.isna(rate):
        return 0
    elif rate >= 256:
        return 4.5
    elif rate >= 192:
        return 4
    elif rate >= 128:
        return 3
    elif rate >= 96:
        return 2
    else:
        return 1
elif codec == 'MP3':
    if pd.isna(rate):
        return 0
    elif rate >= 320:
        return 4
    elif rate >= 192:
        return 3
    elif rate >= 128:
        return 2
    elif rate >= 64:
        return 1
    else:
        return 0.5
else:
    return 0

df_score['编码得分'] = df_score.apply(codec_score, axis=1)

# Step 4: 归一化评分（直接以三项得分加权归一化）
w_sr, w_bd, w_codec = 0.4, 0.2, 0.4
raw_total_score = (
    w_sr * df_score['采样率得分'] +
    w_bd * df_score['位深得分'] +
    w_codec * df_score['编码得分']
)

```

```

df_score['音质评分'] = ((raw_total_score - raw_total_score.min()) /
(raw_total_score.max() - raw_total_score.min())).round(3)

# Step 5: 排序并筛选类型
df_sorted = df_score.sort_values(by='音质评分',
ascending=False).reset_index(drop=True)
df_voice = df_sorted[df_sorted['类型'] == '语音'].reset_index(drop=True)
df_music = df_sorted[df_sorted['类型'] == '音乐'].reset_index(drop=True)

# Step 6: 保存两个输出文件
df_voice.to_csv('语音音质评分.csv', index=False)
df_music.to_csv('音乐音质评分.csv', index=False)

```

附录 5

介绍：该代码是 Python 编写的（问题 2 代码），作用是计算性价比

```

import pandas as pd
import matplotlib.pyplot as plt
import platform

# 设置中文字体（跨平台兼容）
def set_chinese_font():
    system = platform.system()
    try:
        if system == 'Windows':
            plt.rcParams['font.sans-serif'] = ['SimHei']
        elif system == 'Darwin':
            plt.rcParams['font.sans-serif'] = ['Arial Unicode MS']
        else:
            plt.rcParams['font.sans-serif'] = ['WenQuanYi Zen Hei']
        plt.rcParams['axes.unicode_minus'] = False
    except:
        print("字体设置失败，请手动安装中文字体")

set_chinese_font()

def calculate_scene_adjusted_score(row, scene_type, max_size):
    """根据场景类型调整评分权重"""
    if scene_type == '语音':
        # 语音场景：更重视文件大小（传输效率），权重偏向小文件
        size_weight = 0.7 # 文件大小权重更高
        quality_weight = 0.3
    else: # 音乐场景
        # 音乐场景：更重视音质，权重偏向高音质
        size_weight = 0.3

```

```

        quality_weight = 0.7

        # 标准化文件大小（越小越好，使用 1 - 标准化值）
        normalized_size = 1 - (row['文件大小(字节)'] / max_size)

        # 综合评分 = 音质评分×音质权重 + 标准化大小×大小权重
        return (row['音质评分'] * quality_weight) + (normalized_size * size_weight)

def analyze_audio_data(csv_file, scene_type):
    df = pd.read_csv(csv_file)

    # 基础性价比（通用）
    df['基础性价比'] = (df['音质评分'] / df['文件大小(字节)']) * 1e6

    # 计算当前数据框的最大文件大小（用于标准化）
    max_size = df['文件大小(字节)'].max()

    # 场景适配性价比（根据场景类型调整权重）
    df['类型适配评分'] = df.apply(
        lambda x: calculate_scene_adjusted_score(x, scene_type, max_size),
        axis=1
    )

    return df # 已移除场景类型标记

def compare_scenes(voice_csv, music_csv):
    # 分析两类数据
    voice_df = analyze_audio_data(voice_csv, '语音')
    music_df = analyze_audio_data(music_csv, '音乐')

    # 合并数据（不再需要场景类型列）
    combined_df = pd.concat([voice_df, music_df])

    # 结果分析
    print("=== 类型特性分析 ===")
    print(f'语音文件平均大小: {voice_df['文件大小(字节)'].mean()/1024:.1f} KB')
    print(f'音乐文件平均大小: {music_df['文件大小(字节)'].mean()/1024:.1f} KB')
    print(f'语音文件平均音质评分: {voice_df['音质评分'].mean():.2f}')
    print(f'音乐文件平均音质评分: {music_df['音质评分'].mean():.2f}\n')

    # 保存推荐方案
    voice_recommended = voice_df.loc[voice_df['类型适配评分'].idxmax()]

```



```

music_recommended = music_df.loc[music_df['类型适配评分'].idxmax()]

print("\n=== 推荐编码方案 ===")
print(f" 语音类型最佳方案： {voice_recommended['文件名']} (评分:{voice_recommended['类型适配评分']:.2f})")
print(f" 音乐类型最佳方案： {music_recommended['文件名']} (评分:{music_recommended['类型适配评分']:.2f})")

return combined_df # 返回不包含场景类型的数据

voice_csv = '语音音质评分.csv'
music_csv = '音乐音质评分.csv'

result_df = compare_scenes(voice_csv, music_csv)
# 保存时确保不包含场景相关列
result_df.to_csv('类型化音频分析结果.csv', index=False)

```

附录 6

介绍：该代码是 Python 编写的（问题 2 代码），作用是音频性价比结果可视化

```

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import platform
from matplotlib.gridspec import GridSpec
import numpy as np

# 中文字体设置（跨平台）
def set_chinese_font():
    system = platform.system()
    try:
        if system == 'Windows':
            plt.rcParams['font.sans-serif'] = ['SimHei']
        elif system == 'Darwin':
            plt.rcParams['font.sans-serif'] = ['Arial Unicode MS']
        else:
            plt.rcParams['font.sans-serif'] = ['WenQuanYi Zen Hei']
        plt.rcParams['axes.unicode_minus'] = False
    except:
        print("字体设置失败，请手动安装中文字体")
    set_chinese_font()

def create_visualizations(voice_df, music_df):
    """创建 6 种专业可视化图表"""

```

列

```
combined_df = pd.concat([voice_df, music_df])
combined_df['类型'] = ['语音']*len(voice_df) + ['音乐']*len(music_df) # 临时

# 1. 性价比雷达图
plt.figure(figsize=(10, 8))
ax2 = plt.subplot(111, polar=True) # 创建极坐标系统
categories = ['音质评分', '基础性价比', '类型适配评分']
stats = combined_df.groupby('类型')[categories].mean()
angles = np.linspace(0, 2*np.pi, len(categories), endpoint=False).tolist()
angles += angles[:1] # 闭合雷达图

for label, color in [('语音', 'blue'), ('音乐', 'red')]:
    values = stats.loc[label].values.tolist()
    values += values[:1]
    ax2.plot(angles, values, color=color, linewidth=2, label=label)
    ax2.fill(angles, values, color=color, alpha=0.25)

ax2.set_xticks(angles[:-1])
ax2.set_xticklabels(categories)
ax2.set_title('各维度均值雷达图对比')
ax2.legend(loc='upper right')
plt.show()

# 2. 文件大小-评分热力图
plt.figure(figsize=(10, 8))
ax4 = plt.gca()
heatmap_data = combined_df.pivot_table(
    index=pd.cut(combined_df['文件大小(字节)'], bins=5),
    columns='类型',
    values='类型适配评分',
    aggfunc='mean'
)
sns.heatmap(
    heatmap_data,
    annot=True,
    fmt=".2f",
    cmap="YlGnBu",
    linewidths=.5,
    ax=ax4
)
ax4.set_title('文件大小分段-类型热力图')
ax4.set_xlabel("")
ax4.set_ylabel('文件大小分段')
```

```

plt.show()

# 3. 平行坐标图
plt.figure(figsize=(10, 8))
ax5 = plt.gca()
sample_df = combined_df.sample(min(15, len(combined_df)),
random_state=42).copy()
# 确保文件名是字符串类型
sample_df['文件名'] = sample_df['文件名'].astype(str)
# 只选择数值型列进行平行坐标分析
pd.plotting.parallel_coordinates(
    sample_df[['类型', '音质评分', '文件大小(字节)', '类型适配评分']],
    '类型',
    color=['blue', 'red'],
    ax=ax5
)
ax5.set_title('多维度平行坐标图（随机采样 15 条）')
ax5.grid(True)
ax5.set_xticklabels(ax5.get_xticklabels(), rotation=45)
plt.show()

voice_csv = '语音音质评分.csv'
music_csv = '音乐音质评分.csv'

# 获取分析数据（不包含类型列）
def analyze_audio_data(csv_file, audio_type):
    df = pd.read_csv(csv_file)
    df['基础性价比'] = (df['音质评分'] / df['文件大小(字节)']) * 1e6
    max_size = df['文件大小(字节)'].max()
    df['类型适配评分'] = df.apply(
        lambda x: calculate_scene_adjusted_score(x, audio_type, max_size),
        axis=1
    )
    return df

def calculate_scene_adjusted_score(row, audio_type, max_size):
    if audio_type == '语音':
        size_weight = 0.7
        quality_weight = 0.3
    else:
        size_weight = 0.3
        quality_weight = 0.7
    normalized_size = 1 - (row['文件大小(字节)'] / max_size)
    return (row['音质评分'] * quality_weight) + (normalized_size * size_weight)

```

```

voice_df = analyze_audio_data(voice_csv, '语音')
music_df = analyze_audio_data(music_csv, '音乐')

# 生成3种可视化图表
create_visualizations(voice_df, music_df)

# 保存结果（不包含类型列）
result_df = pd.concat([voice_df, music_df])
result_df.to_csv('类型化音频分析结果.csv', index=False)

```

附录 7

介绍：该代码是 Python 编写的（问题 3 代码）

```

import os
import librosa
import numpy as np
import pandas as pd
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from pydub import AudioSegment
AudioSegment.ffmpeg = r"D:\学科网啊\ffmpeg\bin\ffmpeg.exe"

# ----- 配置路径和依赖 -----
# 原始音频路径（根据实际路径修改）
base_path = r"D:\十五届妈妈杯\2025 年第十五届 MathorCup 数学应用挑战赛赛
题\C 题\附件 1"
music_path = os.path.join(base_path, "原始音乐_48kHz_24bit.wav")
speech_path = os.path.join(base_path, "原始语音_48kHz_24bit.wav")

# ----- 步骤 1：特征提取 -----
def extract_advanced_features(audio_path):
    """提取音频特征，MFCC 展开为独立标量特征"""
    y, sr = librosa.load(audio_path, sr=None)

    # 频谱质心
    spectral_centroid = librosa.feature.spectral_centroid(y=y, sr=sr).mean()

    # 动态范围（dB）
    peak = np.max(np.abs(y))
    rms = np.sqrt(np.mean(y**2))
    dynamic_range = 20 * np.log10(peak / rms) if rms != 0 else 0

```

```

# 过零率
zcr = librosa.feature.zero_crossing_rate(y).mean()

# 频谱带宽
spectral_bandwidth = librosa.feature.spectral_bandwidth(y=y, sr=sr).mean()

# MFCC（前 13 个系数，展开为独立特征）
mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13).mean(axis=1)
mfcc_features = {f'mfcc_{i+1}': mfcc[i] for i in range(13)}

return {
    'spectral_centroid': spectral_centroid,
    'dynamic_range': dynamic_range,
    'zcr': zcr,
    'spectral_bandwidth': spectral_bandwidth,
    **mfcc_features
}

# ----- 步骤 2：构建数据集 -----
# 提取特征（假设音乐和语音各 1 个样本）
features_music = extract_advanced_features(music_path)
features_speech = extract_advanced_features(speech_path)

# 转换为标量化数据集（MFCC 展开为 13 列）
X = [
    list(features_music.values()), # 音乐样本
    list(features_speech.values()) # 语音样本
]
y = ['music', 'speech']

# 创建 DataFrame
df_features = pd.DataFrame(X, columns=features_music.keys())
df_features['label'] = y

# ----- 步骤 3：训练分类模型（修正后） -----
# 标准化特征
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df_features.drop('label', axis=1))
y_labels = df_features['label']

# 直接使用所有样本训练（跳过测试集划分）
clf = SVC(kernel='rbf', C=1.0, probability=True)
clf.fit(X_scaled, y_labels) # X_scaled 包含全部 2 个样本

```

```

# ----- 步骤 4: 自适应编码 -----
def adaptive_encode(audio_path, clf, scaler):
    """自适应选择编码参数并保存文件"""
    # 提取特征
    features = extract_advanced_features(audio_path)
    feature_vector = np.array(list(features.values())).reshape(1, -1)
    feature_scaled = scaler.transform(feature_vector)

    # 分类预测
    audio_type = clf.predict(feature_scaled)[0]
    proba = clf.predict_proba(feature_scaled)[0]
    print(f'音频类型: {audio_type} (置信度: {np.max(proba):.2f})')

    # 根据类型选择参数
    if audio_type == 'speech':
        # 语音: 低采样率、高压缩
        target_format = "mp3"
        bitrate = "64k"
        output_sr = 16000
    else:
        # 音乐: 高音质、低压缩
        target_format = "flac"
        bitrate = "320k"
        output_sr = 44100

    # 编码并保存
    audio = AudioSegment.from_wav(audio_path)
    audio = audio.set_frame_rate(output_sr)
    output_path = audio_path.replace(".wav", f"_adaptive.{target_format}")
    audio.export(output_path, format=target_format, bitrate=bitrate)
    return output_path

# ----- 步骤 5: 性能评估 -----
def calculate_snr(original_path, encoded_path):
    """计算信噪比 (SNR) """
    y_orig, sr_orig = librosa.load(original_path, sr=None)
    y_enc, sr_enc = librosa.load(encoded_path, sr=None)

    # 对齐长度
    min_len = min(len(y_orig), len(y_enc))
    y_orig = y_orig[:min_len]
    y_enc = y_enc[:min_len]

```

```

# 计算 SNR
noise = y_orig - y_enc
snr = 10 * np.log10(np.sum(y_orig**2) / (np.sum(noise**2) + 1e-10))
return snr

# 处理原始文件并评估
music_encoded = adaptive_encode(music_path, clf, scaler)
music_snr = calculate_snr(music_path, music_encoded)
music_size = os.path.getsize(music_encoded) / 1024 # KB

speech_encoded = adaptive_encode(speech_path, clf, scaler)
speech_snr = calculate_snr(speech_path, speech_encoded)
speech_size = os.path.getsize(speech_encoded) / 1024 # KB

# ----- 步骤 6: 对比固定参数方案 -----
def fixed_encode(audio_path, format="mp3", sr=44100, bitrate="128k"):
    """固定参数编码"""
    audio = AudioSegment.from_wav(audio_path)
    audio = audio.set_frame_rate(sr)
    output_path = audio_path.replace(".wav", f"_fixed.{format}")
    audio.export(output_path, format=format, bitrate=bitrate)
    return output_path

# 固定参数编码（统一使用 MP3 128kbps）
music_fixed = fixed_encode(music_path)
music_fixed_snr = calculate_snr(music_path, music_fixed)
music_fixed_size = os.path.getsize(music_fixed) / 1024

speech_fixed = fixed_encode(speech_path)
speech_fixed_snr = calculate_snr(speech_path, speech_fixed)
speech_fixed_size = os.path.getsize(speech_fixed) / 1024

```

附录 8

介绍：该代码是 Python 编写的（问题 4 代码），作用是识别音频噪声类型（part1.wav）并可视化结果

```

import numpy as np
import librosa
import librosa.display
import matplotlib.pyplot as plt
import matplotlib.patches as patches
import csv
from scipy.signal import medfilt
import IPython.display as ipd

```

```

import platform
from scipy.stats import entropy

# 中文字体设置 (跨平台)
def set_chinese_font():
    system = platform.system()
    try:
        if system == 'Windows':
            plt.rcParams['font.sans-serif'] = ['SimHei']
        elif system == 'Darwin':
            plt.rcParams['font.sans-serif'] = ['Arial Unicode MS']
        else:
            plt.rcParams['font.sans-serif'] = ['WenQuanYi Zen Hei']
        plt.rcParams['axes.unicode_minus'] = False
    except:
        print("字体设置失败，请手动安装中文字体")
set_chinese_font()

def compute_spectral_entropy(y, sr, n_fft=2048, hop_length=512):
    S = np.abs(librosa.stft(y, n_fft=n_fft, hop_length=hop_length))
    psd = S / np.sum(S, axis=0, keepdims=True) # 归一化为概率分布
    entropy_vals = entropy(psd, base=2, axis=0) # 对每帧计算熵
    return np.mean(entropy_vals)

# ===== 参数设置 =====
audio_path = "附件 2/part1.wav"
segment_duration = 2.0 # 每段时长 (秒)
output_csv = "part1 噪声识别结果.csv" # 输出结果文件

# ===== 1. 读取音频 =====
y, sr = librosa.load(audio_path, sr=None)
total_duration = len(y) / sr
segment_samples = int(segment_duration * sr)
num_segments = int(np.ceil(len(y) / segment_samples))
print(f"音频总长: {total_duration:.2f}s, 分为 {num_segments} 个 {segment_duration}s 片段")

# ===== 2. 改进的语音检测 =====
def is_voice_segment(segment, sr):
    """基于梅尔频谱熵和能量的语音检测"""
    mfcc = librosa.feature.mfcc(y=segment, sr=sr, n_mfcc=13)
    spectral_entropy = compute_spectral_entropy(segment, sr)
    energy = np.mean(segment**2)
    return (spectral_entropy > 0.6) and (energy > 0.001)

```



```

# ===== 3. 改进的特征提取 =====
def extract_features(segment, sr):
    """提取噪声特征"""
    D = librosa.stft(segment, n_fft=1024)
    S = np.abs(D)
    freqs = librosa.fft_frequencies(sr=sr, n_fft=1024)

    # 频率维度掩码
    freq_mask = np.ones(S.shape[0], dtype=bool)
    base_freqs = [80, 160, 240, 320]
    for f in base_freqs:
        idx = np.argmin(np.abs(freqs - f))
        freq_mask[max(0, idx-1):idx+2] = False

    S_noise = S[freq_mask, :]

    if S_noise.shape[1] < 2: # 不能 diff
        burst = 0
    else:
        burst = np.std(np.diff(np.mean(S_noise, axis=0))) # 时域波动

    band = np.std(np.mean(S_noise, axis=1)) # 频域集中度
    bg = np.mean(np.var(S_noise, axis=1)) # 背景均匀度
    return burst, band, bg

# ===== 4. 信噪比估算 =====
def estimate_snr(segment):
    """基于谱减法的 SNR 估算"""
    D = librosa.stft(segment, n_fft=1024)
    S = np.abs(D)
    noise_profile = np.percentile(S, 10, axis=1) # 噪声底噪估计
    signal_power = np.sum(S**2)
    noise_power = np.sum(noise_profile**2)
    return 10 * np.log10(signal_power / (noise_power + 1e-10)) # 避免除零

# ===== 5. 动态阈值计算 =====
def calculate_thresholds(y, sr, segment_samples, num_segments):
    """基于非语音段计算自适应阈值"""
    bursts, bands = [], []
    for i in range(num_segments):
        segment = y[i*segment_samples : (i+1)*segment_samples]
        if not is_voice_segment(segment, sr):
            burst, band, _ = extract_features(segment, sr)

```

```

        bursts.append(burst)
        bands.append(band)
    return {
        'burst': np.percentile(bursts, 70),
        'band': np.percentile(bands, 70),
    }

thresholds = calculate_thresholds(y, sr, segment_samples, num_segments)

# ===== 6. 精确噪声分类 =====
def classify_noise(segment, sr):
    """结合频段特征和谐波检测的分类"""
    burst, band, bg = extract_features(segment, sr)

    # 带状噪声需满足: 1) 频带集中度高 2) 无谐波结构
    if band > thresholds['band']:
        peaks = librosa.util.peak_pick(
            x=np.mean(np.abs(librosa.stft(segment)), axis=0),
            pre_max=2,
            post_max=2,
            pre_avg=2,
            post_avg=2,
            delta=0.5,
            wait=0
        )

        if len(peaks) <= 1: # 真带状噪声只有1个主峰
            return "带状噪声", burst, band, bg

    if burst > thresholds['burst']:
        return "突发噪声", burst, band, bg

    return "背景噪声", burst, band, bg

# ===== 7. 主分析流程 =====
results = []
snr_values = []
for i in range(num_segments):
    start = i * segment_samples
    end = min((i + 1) * segment_samples, len(y))
    segment = y[start:end]

    noise_type, burst, band, bg = classify_noise(segment, sr)

```

```

# 记录结果
snr = estimate_snr(segment)
snr_values.append(snr)
results.append([round(i * segment_duration, 2),
                round(min((i + 1) * segment_duration, total_duration), 2),
                snr,
                noise_type,
                burst,
                band,
                bg])

# ===== 8. 结果保存与可视化 =====
# 保存CSV
with open(output_csv, "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["起始时间(s)", "结束时间(s)", "信噪比(dB)", "噪声类型", "突发噪声", "带状噪声", "背景噪声"])
    writer.writerows(results)

# 绘制时频图
D = librosa.stft(y, n_fft=1024)
S_db = librosa.amplitude_to_db(np.abs(D), ref=np.max)

plt.figure(figsize=(9, 6))
# 设置横坐标范围为0到70秒
x_coords = np.linspace(0, 70, S_db.shape[1])
librosa.display.specshow(S_db, sr=sr, x_coords=x_coords, x_axis='time',
y_axis='hz')
plt.xlim(0, 70) # 确保横坐标范围正确
plt.colorbar(format='%+2.0f dB')

# 标注噪声区间
colors = {'语音': 'lightgray', '带状噪声': 'dodgerblue', '突发噪声': 'darkred', '背景噪声': 'darkgreen'}
for start, end, _, noise_type, _, _ in results:
    if start < 70 and end > 0:
        start = max(0, start)
        end = min(70, end)
        plt.axvspan(start, end, color=colors[noise_type], alpha=0.4) # 适度调整透明度

# 统计结果
noise_stats = {}
for _, _, _, noise_type, _, _ in results:

```

```

noise_stats[noise_type] = noise_stats.get(noise_type, 0) + segment_duration

print("\n=== 噪声统计 ===")
for typ, duration in noise_stats.items():
    print(f'{typ}: {duration:.1f}s ({duration/total_duration:.1%})')

# 饼图可视化噪声类型比例
plt.figure(figsize=(6, 6))
labels = list(noise_stats.keys())
sizes = [noise_stats[k] for k in labels]
colors_pie = ['#66c2a5', '#fc8d62', '#8da0cb', '#e78ac3'] # 更加丰富的颜色
plt.pie(sizes, labels=labels, colors=colors_pie, autopct='%1.1f%%', startangle=140,
textprops={'fontsize': 14})
plt.title("part1.wav 噪声类型分布", fontsize=20) # 更大字号
plt.axis('equal')
plt.show()

```

附录 9

介绍：该代码是 Python 编写的（问题 4 代码），作用是识别音频噪声类型（part2.wav）并可视化结果

```

import numpy as np
import librosa
import librosa.display
import matplotlib.pyplot as plt
import matplotlib.patches as patches
import csv
from scipy.signal import medfilt
import IPython.display as ipd
import platform
from scipy.stats import entropy

# 中文字体设置（跨平台）
def set_chinese_font():
    system = platform.system()
    try:
        if system == 'Windows':
            plt.rcParams['font.sans-serif'] = ['SimHei']
        elif system == 'Darwin':
            plt.rcParams['font.sans-serif'] = ['Arial Unicode MS']
        else:
            plt.rcParams['font.sans-serif'] = ['WenQuanYi Zen Hei']
        plt.rcParams['axes.unicode_minus'] = False
    except:

```

```

        print("字体设置失败，请手动安装中文字体")
set_chinese_font()

def compute_spectral_entropy(y, sr, n_fft=2048, hop_length=512):
    S = np.abs(librosa.stft(y, n_fft=n_fft, hop_length=hop_length))
    psd = S / np.sum(S, axis=0, keepdims=True) # 归一化为概率分布
    entropy_vals = entropy(psd, base=2, axis=0) # 对每帧计算熵
    return np.mean(entropy_vals)

# ===== 参数设置 =====
audio_path = "附件 2/part2.wav"
segment_duration = 2.0 # 每段时长 (秒)
output_csv = "part2 噪声识别结果.csv" # 输出结果文件

# ===== 1. 读取音频 =====
y, sr = librosa.load(audio_path, sr=None)
total_duration = len(y) / sr
segment_samples = int(segment_duration * sr)
num_segments = int(np.ceil(len(y) / segment_samples))
print(f"音频总长： {total_duration:.2f}s，分为 {num_segments} 个 {segment_duration}s 片段")

# ===== 2. 改进的语音检测 =====
def is_voice_segment(segment, sr):
    """基于梅尔频谱熵和能量的语音检测"""
    mfcc = librosa.feature.mfcc(y=segment, sr=sr, n_mfcc=13)
    spectral_entropy = compute_spectral_entropy(segment, sr)
    energy = np.mean(segment**2)
    return (spectral_entropy > 0.6) and (energy > 0.004)

# ===== 3. 改进的特征提取 =====
def extract_features(segment, sr):
    """提取噪声特征"""
    D = librosa.stft(segment, n_fft=1024)
    S = np.abs(D)
    freqs = librosa.fft_frequencies(sr=sr, n_fft=1024)

    # 频率维度掩码
    freq_mask = np.ones(S.shape[0], dtype=bool)
    base_freqs = [80, 160, 240, 320]
    for f in base_freqs:
        idx = np.argmin(np.abs(freqs - f))
        freq_mask[max(0, idx-1):idx+2] = False

```

```

S_noise = S[freq_mask, :]

if S_noise.shape[1] < 2: # 不能diff
    burst = 0
else:
    burst = np.std(np.diff(np.mean(S_noise, axis=0))) # 时域波动

band = np.std(np.mean(S_noise, axis=1)) # 频域集中度
bg = np.mean(np.var(S_noise, axis=1)) # 背景均匀度
return burst, band, bg

# ===== 4. 信噪比估算 =====
def estimate_snr(segment):
    """基于谱减法的 SNR 估算"""
    D = librosa.stft(segment, n_fft=1024)
    S = np.abs(D)
    noise_profile = np.percentile(S, 10, axis=1) # 噪声底噪估计
    signal_power = np.sum(S**2)
    noise_power = np.sum(noise_profile**2)
    return 10 * np.log10(signal_power / (noise_power + 1e-10)) # 避免除零

# ===== 5. 动态阈值计算 =====
def calculate_thresholds(y, sr, segment_samples, num_segments):
    """基于非语音段计算自适应阈值"""
    bursts, bands = [], []
    for i in range(num_segments):
        segment = y[i*segment_samples : (i+1)*segment_samples]
        if not is_voice_segment(segment, sr):
            burst, band, _ = extract_features(segment, sr)
            bursts.append(burst)
            bands.append(band)
    return {
        'burst': np.percentile(bursts, 70),
        'band': np.percentile(bands, 70),
    }

thresholds = calculate_thresholds(y, sr, segment_samples, num_segments)

# ===== 6. 精确噪声分类 =====
def classify_noise(segment, sr):
    """结合频段特征和谐波检测的分类"""
    burst, band, bg = extract_features(segment, sr)

    # 带状噪声需满足: 1) 频带集中度高 2) 无谐波结构

```

```

if band > thresholds['band']:
    peaks = librosa.util.peak_pick(
        x=np.mean(np.abs(librosa.stft(segment)), axis=0),
        pre_max=2,
        post_max=2,
        pre_avg=2,
        post_avg=2,
        delta=0.5,
        wait=0
    )

    if len(peaks) <= 1: # 真带状噪声只有1个主峰
        return "带状噪声", burst, band, bg

if burst > thresholds['burst']:
    return "突发噪声", burst, band, bg

return "背景噪声", burst, band, bg

# ===== 7. 主分析流程 =====
results = []
snr_values = []
for i in range(num_segments):
    start = i * segment_samples
    end = min((i + 1) * segment_samples, len(y))
    segment = y[start:end]

    noise_type, burst, band, bg = classify_noise(segment, sr)

    # 记录结果
    snr = estimate_snr(segment)
    snr_values.append(snr)
    results.append([round(i * segment_duration, 2),
                    round(min((i + 1) * segment_duration, total_duration), 2),
                    snr,
                    noise_type,
                    burst,
                    band,
                    bg])

# ===== 8. 结果保存与可视化 =====
# 保存CSV
with open(output_csv, "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)

```

```

writer.writerow(["起始时间(s)", "结束时间(s)", "信噪比(dB)", "噪声类型", "突
发噪声", "带状噪声", "背景噪声"])
writer.writerows(results)

# 绘制时频图
D = librosa.stft(y, n_fft=1024)
S_db = librosa.amplitude_to_db(np.abs(D), ref=np.max)

plt.figure(figsize=(9, 6))
# 设置横坐标范围为0 到70 秒
x_coords = np.linspace(0, 70, S_db.shape[1])
librosa.display.specshow(S_db, sr=sr, x_coords=x_coords, x_axis='time',
y_axis='hz')
plt.xlim(0, 70) # 确保横坐标范围正确
plt.colorbar(format='%+2.0f dB')

# 标注噪声区间
colors = {'语音': 'lightgray', '带状噪声': 'dodgerblue', '突发噪声': 'darkred', '背景噪
声': 'darkgreen'}
for start, end, _, noise_type, _, _ in results:
    if start < 70 and end > 0:
        start = max(0, start)
        end = min(70, end)
        plt.axvspan(start, end, color=colors[noise_type], alpha=0.4) # 适度调整透
明度

# 统计结果
noise_stats = {}
for _, _, _, noise_type, _, _ in results:
    noise_stats[noise_type] = noise_stats.get(noise_type, 0) + segment_duration

print("\n=== 噪声统计 ===")
for typ, duration in noise_stats.items():
    print(f'{typ}: {duration:.1f}s ({duration/total_duration:.1%})')

# 饼图可视化噪声类型比例
plt.figure(figsize=(6, 6))
labels = list(noise_stats.keys())
sizes = [noise_stats[k] for k in labels]
colors_pie = ['#66c2a5', '#fc8d62', '#8da0cb', '#e78ac3'] # 更加丰富的颜色
plt.pie(sizes, labels=labels, colors=colors_pie, autopct='%1.1f%%', startangle=140,
textprops={'fontsize': 14})
plt.title("part2.wav 噪声类型分布", fontsize=20) # 更大字号
plt.axis('equal')
plt.show()

```


附录 10

介绍：该代码是 Python 编写的（问题 4 代码），作用是给音频文件（part1.wav）去噪，可视化、输出并保存去噪结果（part1_denoised.wav）

```
import numpy as np
import librosa
import librosa.display
import scipy.signal as signal
import matplotlib.pyplot as plt
import IPython.display as ipd
import soundfile as sf
import platform

# 设置中文字体（跨平台兼容）
def set_chinese_font():
    system = platform.system()
    try:
        if system == 'Windows':
            plt.rcParams['font.sans-serif'] = ['SimHei']
        elif system == 'Darwin':
            plt.rcParams['font.sans-serif'] = ['Arial Unicode MS']
        else:
            plt.rcParams['font.sans-serif'] = ['WenQuanYi Zen Hei']
        plt.rcParams['axes.unicode_minus'] = False
    except:
        print("字体设置失败")

set_chinese_font()
# 读取音频文件
audio_path = "附件 2/part1.wav" # 替换为你的音频路径
y, sr = librosa.load(audio_path, sr=None)

# 设定每段音频的时长
segment_duration = 2.0
segment_samples = int(segment_duration * sr)
num_segments = int(np.ceil(len(y) / segment_samples))

# 去噪策略
def denoise_segment(segment, noise_type, sr):
    """根据噪声类型选择去噪策略"""
    if noise_type == "带状噪声":
        # 带状噪声：带通滤波
        lowcut = 300.0 # 设置低频截止
```

```

        highcut = 3000.0 # 设置高频截止
        nyquist = 0.5 * sr
        low = lowcut / nyquist
        high = highcut / nyquist
        b, a = signal.butter(1, [low, high], btype='band')
        return signal.filtfilt(b, a, segment)

    elif noise_type == "突发噪声":
        # 突发噪声: 中值滤波
        return signal.medfilt(segment, kernel_size=3)

    elif noise_type == "背景噪声":
        # 背景噪声: 谱减法 (简单实现)
        D = librosa.stft(segment, n_fft=1024)
        S = np.abs(D)
        noise_profile = np.percentile(S, 10, axis=1) # 噪声底噪估计
        S_denoised = np.maximum(S - noise_profile[:, np.newaxis], 0) # 简单的谱
        return librosa.istft(S_denoised)

    return segment

# 处理音频
denoised_audio = []
for i in range(num_segments):
    start = i * segment_samples
    end = min((i + 1) * segment_samples, len(y))
    segment = y[start:end]

    # 在这里假设噪声类型是通过某种方式检测出来的, 这里可以根据你的需求设置不同噪声类型
    noise_type = "背景噪声" # 这里可以根据实际的噪声检测结果来调整

    # 去噪处理
    denoised_segment = denoise_segment(segment, noise_type, sr)
    denoised_audio.append(denoised_segment)

# 将去噪后的音频合并
denoised_audio = np.concatenate(denoised_audio)

# 播放去噪后的音频
ipd.display(ipd.Audio(denoised_audio, rate=sr))

# 可视化原音频和去噪后的音频对比

```

```

plt.figure(figsize=(14, 6))

# 原音频波形
plt.subplot(2, 1, 1)
librosa.display.waveshow(y, sr=sr, color='gray')
plt.title("part1.wav 原始音频波形", fontsize=20) # 增加字体大小
plt.xlabel("时间 (秒)", fontsize=15) # 更大字号
plt.ylabel("振幅", fontsize=15) # 更大字号

# 去噪后的音频波形
plt.subplot(2, 1, 2)
librosa.display.waveshow(denoised_audio, sr=sr, color='blue')
plt.title("part1.wav 去噪后的音频波形", fontsize=20) # 增加字体大小
plt.xlabel("时间 (秒)", fontsize=15) # 更大字号
plt.ylabel("振幅", fontsize=15) # 更大字号

plt.tight_layout()
plt.show()

# 保存去噪音频文件
output_path = "part1_denoised.wav"
sf.write(output_path, denoised_audio, sr)
print(f'去噪后的音频已保存至: {output_path}')

import numpy as np
import librosa
import librosa.display
import matplotlib.pyplot as plt
import platform
from scipy.stats import entropy
import soundfile as sf

# 设置中文字体（跨平台兼容）
def set_chinese_font():
    system = platform.system()
    try:
        if system == 'Windows':
            plt.rcParams['font.sans-serif'] = ['SimHei']
        elif system == 'Darwin':
            plt.rcParams['font.sans-serif'] = ['Arial Unicode MS']
        else:
            plt.rcParams['font.sans-serif'] = ['WenQuanYi Zen Hei']
        plt.rcParams['axes.unicode_minus'] = False
    except:

```

```

print("字体设置失败")

set_chinese_font()

# === 读取去噪前音频 ===
original_path = "附件 2/part1.wav" # 去噪前音频
y_original, sr = librosa.load(original_path, sr=None)
total_duration = len(y_original) / sr

# === 读取去噪后的音频 ===
denoised_path = "part1_denoised.wav" # 去噪后的音频
y_denoised, sr = librosa.load(denoised_path, sr=None)

# === 设置参数 ===
segment_duration = 2.0
segment_samples = int(segment_duration * sr)
num_segments = int(np.ceil(len(y_original) / segment_samples))

# === SNR 估算函数 ===
def estimate_snr(segment):
    D = librosa.stft(segment, n_fft=1024)
    S = np.abs(D)
    noise_profile = np.percentile(S, 10, axis=1)
    signal_power = np.sum(S**2)
    noise_power = np.sum(noise_profile**2)
    return 10 * np.log10(signal_power / (noise_power + 1e-10))

# === 计算去噪前和去噪后的信噪比 ===
snr_values_original = []
snr_values_denoised = []

for i in range(num_segments):
    start = i * segment_samples
    end = min((i + 1) * segment_samples, len(y_original))

    segment_original = y_original[start:end]
    segment_denoised = y_denoised[start:end]

    snr_original = estimate_snr(segment_original)
    snr_denoised = estimate_snr(segment_denoised)

    snr_values_original.append(snr_original)
    snr_values_denoised.append(snr_denoised)

```

```

# === 可视化信噪比 ===
time_axis = np.linspace(0, total_duration, num_segments)
avg_snr_original = np.mean(snr_values_original)
avg_snr_denoised = np.mean(snr_values_denoised)

plt.figure(figsize=(14, 6))

# 绘制去噪前和去噪后的信噪比曲线
plt.plot(time_axis, snr_values_original, color='tomato', label='去噪前信噪比(dB)',
linewidth=2)
plt.plot(time_axis, snr_values_denoised, color='seagreen', label='去噪后信噪比
(dB)', linewidth=2)

# 绘制信噪比平均值
plt.axhline(avg_snr_original, color='red', linestyle='--', label=f'去噪前平均 SNR:
{avg_snr_original:.2f} dB', linewidth=2)
plt.axhline(avg_snr_denoised, color='green', linestyle='--', label=f'去噪后平均 SNR:
{avg_snr_denoised:.2f} dB', linewidth=2)

# 美化图表
plt.xlabel('时间 (秒)', fontsize=18) # 更大字号
plt.ylabel('信噪比 (dB)', fontsize=18) # 更大字号
plt.title('part1.wav 去噪前后信噪比随时间变化', fontsize=20) # 更大字号
plt.legend(fontsize=16) # 更大字号
plt.grid(True, which='both', linestyle='--', linewidth=0.5) # 添加网格线，设置为虚
线

plt.tight_layout()
plt.show()

```

附录 11

介绍：该代码是 Python 编写的（问题 4 代码），作用是给音频文件（part2.wav）去噪，可视化、输出并保存去噪结果（part2_denoised.wav）

```

import numpy as np
import librosa
import librosa.display
import scipy.signal as signal
import matplotlib.pyplot as plt
import IPython.display as ipd
import soundfile as sf

# 读取音频文件
audio_path = "附件 2/part2.wav" # 替换为你的音频路径
y, sr = librosa.load(audio_path, sr=None)

```

```

# 设定每段音频的时长
segment_duration = 2.0
segment_samples = int(segment_duration * sr)
num_segments = int(np.ceil(len(y) / segment_samples))

# 去噪策略
def denoise_segment(segment, noise_type, sr):
    """根据噪声类型选择去噪策略"""
    if noise_type == "带状噪声":
        # 带状噪声：带通滤波
        lowcut = 300.0 # 设置低频截止
        highcut = 3000.0 # 设置高频截止
        nyquist = 0.5 * sr
        low = lowcut / nyquist
        high = highcut / nyquist
        b, a = signal.butter(1, [low, high], btype='band')
        return signal.filtfilt(b, a, segment)

    elif noise_type == "突发噪声":
        # 突发噪声：中值滤波
        return signal.medfilt(segment, kernel_size=3)

    elif noise_type == "背景噪声":
        # 背景噪声：谱减法（简单实现）
        D = librosa.stft(segment, n_fft=1024)
        S = np.abs(D)
        noise_profile = np.percentile(S, 10, axis=1) # 噪声底噪估计
        S_denoised = np.maximum(S - noise_profile[:, np.newaxis], 0) # 简单的谱
        return librosa.istft(S_denoised)

    return segment

# 处理音频
denoised_audio = []
for i in range(num_segments):
    start = i * segment_samples
    end = min((i + 1) * segment_samples, len(y))
    segment = y[start:end]

    # 在这里假设噪声类型是通过某种方式检测出来的，这里可以根据你的需求设置不同噪声类型
    noise_type = "背景噪声" # 这里可以根据实际的噪声检测结果来调整

```

```

    # 去噪处理
    denoised_segment = denoise_segment(segment, noise_type, sr)
    denoised_audio.append(denoised_segment)

# 将去噪后的音频合并
denoised_audio = np.concatenate(denoised_audio)

# 播放去噪后的音频
ipd.display(ipd.Audio(denoised_audio, rate=sr))

# 可视化原音频和去噪后的音频对比
plt.figure(figsize=(14, 6))

# 原音频波形
plt.subplot(2, 1, 1)
librosa.display.waveshow(y, sr=sr, color='gray')
plt.title("part2.wav 原始音频波形", fontsize=20) # 增加字体大小
plt.xlabel("时间 (秒)", fontsize=15) # 更大字号
plt.ylabel("振幅", fontsize=15) # 更大字号

# 去噪后的音频波形
plt.subplot(2, 1, 2)
librosa.display.waveshow(denoised_audio, sr=sr, color='blue')
plt.title("part2.wav 去噪后的音频波形", fontsize=20) # 增加字体大小
plt.xlabel("时间 (秒)", fontsize=15) # 更大字号
plt.ylabel("振幅", fontsize=15) # 更大字号

plt.tight_layout()
plt.show()

# 保存去噪音频文件
output_path = "part2_denoised.wav"
sf.write(output_path, denoised_audio, sr)
print(f'去噪后的音频已保存至: {output_path}')

import numpy as np
import librosa
import librosa.display
import matplotlib.pyplot as plt
import platform
from scipy.stats import entropy
import soundfile as sf

```

```

# 设置中文字体（跨平台兼容）
def set_chinese_font():
    system = platform.system()
    try:
        if system == 'Windows':
            plt.rcParams['font.sans-serif'] = ['SimHei']
        elif system == 'Darwin':
            plt.rcParams['font.sans-serif'] = ['Arial Unicode MS']
        else:
            plt.rcParams['font.sans-serif'] = ['WenQuanYi Zen Hei']
        plt.rcParams['axes.unicode_minus'] = False
    except:
        print("字体设置失败")

set_chinese_font()

# === 读取去噪前音频 ===
original_path = "附件 2/part2.wav" # 去噪前音频
y_original, sr = librosa.load(original_path, sr=None)
total_duration = len(y_original) / sr

# === 读取去噪后的音频 ===
denoised_path = "part2_denoised.wav" # 去噪后的音频
y_denoised, sr = librosa.load(denoised_path, sr=None)

# === 设置参数 ===
segment_duration = 2.0
segment_samples = int(segment_duration * sr)
num_segments = int(np.ceil(len(y_original) / segment_samples))

# === SNR 估算函数 ===
def estimate_snr(segment):
    D = librosa.stft(segment, n_fft=1024)
    S = np.abs(D)
    noise_profile = np.percentile(S, 10, axis=1)
    signal_power = np.sum(S**2)
    noise_power = np.sum(noise_profile**2)
    return 10 * np.log10(signal_power / (noise_power + 1e-10))

# === 计算去噪前和去噪后的信噪比 ===
snr_values_original = []
snr_values_denoised = []

for i in range(num_segments):

```



```

start = i * segment_samples
end = min((i + 1) * segment_samples, len(y_original))

segment_original = y_original[start:end]
segment_denoised = y_denoised[start:end]

snr_original = estimate_snr(segment_original)
snr_denoised = estimate_snr(segment_denoised)

snr_values_original.append(snr_original)
snr_values_denoised.append(snr_denoised)

# === 可视化信噪比 ===
time_axis = np.linspace(0, total_duration, num_segments)
avg_snr_original = np.mean(snr_values_original)
avg_snr_denoised = np.mean(snr_values_denoised)

plt.figure(figsize=(14, 6))

# 绘制去噪前和去噪后的信噪比曲线
plt.plot(time_axis, snr_values_original, color='tomato', label='去噪前信噪比(dB)',
linewidth=2)
plt.plot(time_axis, snr_values_denoised, color='seagreen', label='去噪后信噪比
(dB)', linewidth=2)

# 绘制信噪比平均值
plt.axhline(avg_snr_original, color='red', linestyle='--', label=f'去噪前平均 SNR:
{avg_snr_original:.2f} dB', linewidth=2)
plt.axhline(avg_snr_denoised, color='green', linestyle='--', label=f'去噪后平均 SNR:
{avg_snr_denoised:.2f} dB', linewidth=2)

# 美化图表
plt.xlabel('时间 (秒)', fontsize=18) # 更大字号
plt.ylabel('信噪比 (dB)', fontsize=18) # 更大字号
plt.title('part2.wav 去噪前后信噪比随时间变化', fontsize=20) # 更大字号
plt.legend(fontsize=16) # 更大字号
plt.grid(True, which='both', linestyle='--', linewidth=0.5) # 添加网格线, 设置为虚
线

plt.tight_layout()
plt.show()

```